HOOX TRADING PLATFORM

DEVOPS INFRASTRUCTURE, BINDINGS & SYSTEM RUNBOOKS

Generated: 2026-05-19

Classification: Technical Documentation Spec

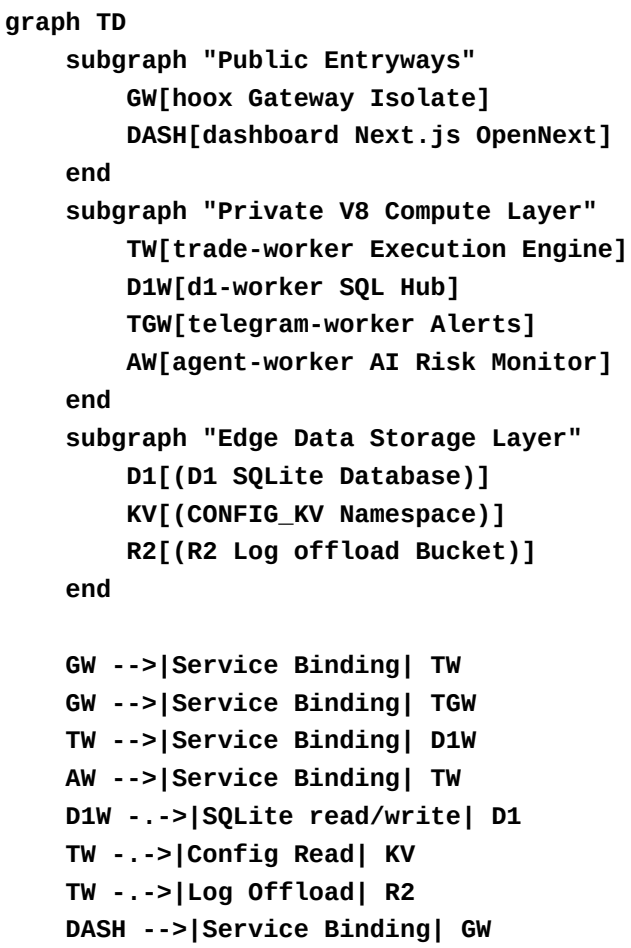
Design Style: Monospace Terminal Console layout

TABLE OF CONTENTS

[S E C T I O N : þ D E V O P S M A N U A L]

&™þ D E V O P S M A N U A L

> **Welcome to the Hoox DevOps & System Operations Manual.** This manual is the primary, production-grade reference for system administrators, security engineers, and platform operators responsible for provisioning edge databases, orchestrating secure V8 service bindings, deploying multi-exchange execution workers, auditing secrets, and monitoring system health.



Ø=Ýúþ O P E R A T O R ' S S Y S T E M D I R E C T O R Y

The DevOps manual is structured into five core architectural operational layers:

1 . Ø=ÜË O P E R A T I O N S & S E T U P R U N B O O K S

Complete guides for bootstrapping environments, managing interactive terminal dashboards, and diagnosing local runner configurations:

- **[Operations & Troubleshooting](setup_and_operations.md)** — Core operations manual, 31-key environment variable matrices, and diagnostic codes.

- **[Core Installation Flow](installation-flow.md)** — Guided, step-by-step local machine and edge provision setup.
- **[Terminal UI Cockpit Development](tui.md)** — Code architectures, stores, and operations of the OpenTUI monitor.

2. Ø<ß×þ A R C H I T E C T U R A L S P E C I F I C A T I O N S

Deep dives into latency structures, isolation parameters, and bindings linkages:

- **[System Topology Overview](architecture/overview.md)** — Architectural layout, regional edge clustering, and scaling limits.
- **[Isolate Communication](architecture/communication.md)** — Deep dive into Service Bindings, zero-TCP routing, and V8 engines.
- **[Data Flow Architecture](architecture/data-flow.md)** — Flowcharts for trade execution, backup queues, and cron risk monitoring.
- **[Bindings Matrix](architecture/bindings.md)** — Exact mapping of D1, KV, Queues, R2, and Service Bindings.
- **[Storage Engineering](architecture/storage.md)** — Persistent storage boundaries, SQLite DDL parameters, and R2 buckets.
- **[Internal Endpoints Map](architecture/endpoints.md)** — Sub-millisecond binding paths and routing maps.
- **[Visual Tokens & Design System](architecture/design-system.md)** — Monochromatic token mappings and visual SVGs catalog.

3. &™þ E D G E W O R K E R M I C R O S E R V I C E S (1 0 P R O F I L E S

Individual developer profiles for each running isolate, cataloging bindings, custom middlewares, and API formats:

- **[hoox Gateway](workers/hoox.md)** · **[trade-worker](workers/trade-worker.md)** · **[agent-worker](workers/agent-worker.md)** · **[telegram-worker](workers/telegram-worker.md)**
- **[d1-worker](workers/d1-worker.md)** · **[web3-wallet-worker](workers/web3-wallet-worker.md)** · **[email-worker](workers/email-worker.md)** · **[analytics-worker](workers/analytics-worker.md)**
- **[report-worker](workers/report-worker.md)** · **[dashboard (Next.js OpenNext)](workers/dashboard.md)**

4. Ø=Þ¢ D E P L O Y M E N T , W A F , & C I / C D P I P E L I N E S

Rollout manuals, Access gates, and GitHub Actions telemetry:

- **[Production Deployment](deployment/production.md)** — Wrangler commands, Account provisioning, and production variables.
- **[CI/CD Workflow pipelines](deployment/cicd.md)** — GitHub Actions secrets, syntax

tests, and automated edge uploads.
- **[Monitoring & Telemetry](deployment/monitoring.md)** — Live wrangler logs streaming and custom Analytics Engine metrics.
- **[Cloudflare Zero Trust Corridor](deployment/zero-trust.md)** — Setting up Access client corridors, IP firewalls, and WAF rules.

Ø=Ü» 5. DEVELOPER & API REFERENCE

TypeScript interfaces, compiler settings, Bun test specs, and HTTP schemas:

- **[Wrangler Dev Setup](development/local-dev.md)** · **[Testing Standards](development/testing.md)** · **[Debugging Runbook](development/debugging.md)**
- **[Exposed API Routes](api/endpoints.md)** · **[Request Payloads](api/payloads.md)** · **[Standard Responses](api/responses.md)**
- **[CLI Commands Engine](cli_features.md)** — Command-line argument parsing, binary execution, and JSON flags.

> **Tip:** First time deploying a Hoox workspace to production? Start with the **[Production Deployment Manual](deployment/production.md)** to verify your Cloudflare Account permissions and execute sequencial deployments seamlessly.

Ø=Ý QUICK LINKS & OFFLINE REFERENCE

- **[End-User Documentation Hub](../home.md)** — Standard setup, cURL webhooks, and TradingView Pine Scripts.
- **[Hoox Git Submodules](https://github.com/jango-blockchained/hoox-setup)** — Central monorepo codebase.
- **[Download Enduser Full PDF Manual](/hoox-setup/Enduser-Full-Documentation.pdf)** — Complete concatenated offline guide.
- **[Download DevOps Full PDF Manual](/hoox-setup/DevOps-Full-Documentation.pdf)** — Complete concatenated offline DevOps spec.
- **[View Consolidated LLM Context Text](/hoox-setup/llm.txt)** — Giant single-file text format for AI/LLM models.

[SECTION: OPERATIONS & RUNBOOK]

Ø=ÜÖ O P E R A T I O N S & R U N B O O K

This manual is the primary, production-grade operations runbook for Hoox administrators. It outlines the complete system topology, toolchain prerequisites, environment variables, sequential deployment protocols, and guided troubleshooting matrices.

---

1. COMPLETE ENVIRONMENT VARIABLES MATRIX (31 KEYS)

These variables are defined in `.env.local` for local building/deploying or injected as encrypted **Workers Secrets** for runtime isolate compute.

A. CORE PLATFORM & INFRASTRUCTURE

- `CLOUDFLARE_API_TOKEN`: Cloudflare API Token with Workers, D1, and KV read/write permissions.
- `CLOUDFLARE_ACCOUNT_ID`: Your unique 32-character Cloudflare dashboard account hash.
- `SUBDOMAIN_PREFIX`: Subdomain prefix under which your public gateway routes compile.
- `NODE_ENV`: Target environment profile: `development`, `staging`, or `production`.
- `HOOX_API_URL`: Local API target URL (automatically configured during dev runs).

B. EXCHANGE API CREDENTIALS (ENCRYPTED SECRETS)

- `BYBIT_API_KEY` & `BYBIT_API_SECRET`: Bybit order placement account key and HMAC-SHA256 signature key.
- `BINANCE_API_KEY` & `BINANCE_API_SECRET`: Binance trade permission key and private HMAC signature.
- `MEXC_API_KEY` & `MEXC_API_SECRET`: MEXC trade permission key and private HMAC signature.

C. TELEGRAM BOT ALERTS & TELEMETRY

- `TELEGRAM_BOT_TOKEN`: Telegram bot token from `@BotFather`.
- `TELEGRAM_CHAT_ID`: Authorized numeric Chat ID for pushed fills and commands.

D. MULTI-PROVIDER AI CREDENTIALS

- `OPENAI_API_KEY`: OpenAI API access key.
- `ANTHROPIC_API_KEY`: Anthropic Claude API access key.
- `GOOGLE_AI_API_KEY`: Google Gemini API access key.

E. DEFI & WEB3 WALLET SETTINGS

- `ETH_MNEMONIC`: Secure 12 or 24-word seed phrase for EVM swaps.
- `RPC_PROVIDER_URL`: HTTP Ethereum / EVM JSON-RPC provider (e.g. Infura/Alchemy).

F. EMAIL PARSING INBOX CONNECTION

- `EMAIL_HOST`: POP3/IMAP mailbox server address.
- `EMAIL_USER`: Target signal mailbox email address.
- `EMAIL_PASS`: Secure app password for mailbox access.

---

2. WORKER DEPLOYMENT SEQUENCE

Because Hoox microservices communicate internally using fast-path **Service Bindings**, they have strict compile-time and deploy-time dependencies. Deploy workers in the following strict sequential hierarchy:

```
  1. analytics-worker    (No dependencies)
  2. report-worker       (Depends on: analytics-worker)
  3. d1-worker           (Depends on: analytics-worker)
  4. telegram-worker     (Depends on: trade-worker, hoox, analytics-worker)
  5. web3-wallet-worker  (Depends on: telegram-worker, analytics-worker)
  6. email-worker        (Depends on: trade-worker, analytics-worker)
  7. trade-worker        (Depends on: d1-worker, telegram-worker, analytics-worker)
  8. agent-worker        (Depends on: d1-worker, trade-worker, telegram-worker,
analytics-worker)
  9. hoox Gateway        (Depends on: trade-worker, telegram-worker, analytics-worker)
  10. dashboard          (Depends on: all services being live)

  # Automated Sequenced Deployment via CLI
  hoox deploy all --auto

  # Or deploy a single specific worker
  hoox deploy worker trade-worker
```

---

3. SECRET MANAGEMENT RUNBOOK

Secrets are securely uploaded to Cloudflare's hardware key vaults using the CLI:

```
  # Inject Bybit Credentials
  hoox secrets set BYBIT_API_KEY "your_bybit_key"
  hoox secrets set BYBIT_API_SECRET "your_bybit_secret"

  # Check active secret synchronization on Cloudflare
  hoox secrets check
```

Ø=Þ¨ 4 . G L O B A L K I L L S W I T C H & E M E R G E N C Y O

The **Global Kill Switch** is your emergency brake. Stored inside the sub-millisecond `CONFIG_KV` namespace, flipping this parameter instantly blocks all incoming trade signals globally in under 10 seconds:

```
# View active state of the Kill Switch
hoox monitor kill-switch show

# Emergency HALT - disable all trade execution immediately
hoox monitor kill-switch on

# Resume normal operations
hoox monitor kill-switch off
```

Ø=Þàþ 5 . T R O U B L E S H O O T I N G & D I A G N O S T I C S

ALT ALTERNATE SCREEN BUFFER CLEANUP

If you force-close the terminal and find that your prompt remains garbled, execute a terminal reset:

```
reset
# or
tput reset
```

502 BAD GATEWAY

- **Root Cause**: Service binding target is missing or has crashed.
- **Fix**: Ensure the dependency worker (e.g. `trade-worker`) has been deployed successfully. Run `hoox deploy all` to rebuild all bonds.

503 SERVICE UNAVAILABLE

- **Root Cause**: The Global Kill Switch is active in KV.
- **Fix**: Verify switch state: `hoox monitor kill-switch show`. If safe, restore trading: `hoox monitor kill-switch off`.

Ø=Ý N E X T S T E P S

- **[Astro Docs Site Config](../getting-started/configuration.md)** — Map out your

build-time environment configurations.
- **[Architecture & Edge Topology](architecture/overview.md)** — In-depth architectural outlines and Service Bindings routing maps.

[SECTION: CLOUDFLARE WORKERS SETUP FLOW]

Ø=Þ€ C L O U D F L A R E ® W O R K E R S S E T U P F L O W

This document details the step-by-step installation, bootstrapping, and validation workflows executed by the Hoox CLI during project initialization.

> **Developer Note:** The Hoox CLI enforces strict type validation for all configuration files via the `Config` and `WorkerConfig` TypeScript interfaces defined in `packages/cli/src/core/types.ts`. Avoid using `as any` or type bypasses when extending wrangler parameters to prevent build-time CLI crashes.

Ø<ß×þ I N T E R A C T I V E S E T U P W I Z A R D (` H O O X I N

To start the system bootstrap, run the init wizard from the monorepo root:

```
hoox init
```

The setup wizard guides you through 7 critical onboarding phases:

PHASE 1: TOOLCHAIN DIAGNOSTICS

Probes your machine to verify that essential developer tools are accessible:

- **Bun**: Used for monorepo package management, CLI binaries execution, and fast unit testing.
- **Git**: Used to verify recursive cloning of worker submodules.
- **Wrangler**: Cloudflare's CLI used to login, tail logs, and upload V8 isolates.

PHASE 2: GLOBAL CONFIGURATION MAPPING

Configures your master workspace credentials stored in `.env.local` and `wrangler.jsonc`:

- **Cloudflare API Token**: Generates and checks permissions.
- **Cloudflare Account ID**: Uniquely identifies your hosting space.
- **Subdomain Prefix**: Defines the worker routing domain (e.g. `hoox` routes to `https://hoox.alpha-trading.workers.dev`).

PHASE 3: MICROSERVICE PROFILE SELECTION

Lets you selectively enable or disable specific workers from the `workers/` directory based on your trading intent (e.g., cross-margin futures execution vs. Web3 DeFi wallet swaps). Disabling unnecessary workers saves deployment bandwidth and keeps resource bindings clean.

PHASE 4: SQLITE DATABASE PROVISIONING

Checks if enabled workers require persistent D1 storage. If yes, it creates the database and initializes database schemas:

```
# Done automatically by the wizard
wrangler d1 create trade-data-db
```

PHASE 5: MANIFEST COMPILATION

Consolidates all chosen parameters and writes your central `wrangler.jsonc` file, mapping out variables and bindings for every worker.

PHASE 6: WORKERS SECRETS INJECTION

Secures sensitive credentials (exchange API keys, Telegram tokens, AI API keys) by encrypting and uploading them to Cloudflare as encrypted Workers Secrets.

PHASE 7: INITIAL ROLLOUT

Compiles, lint-checks, type-checks, and deploys all enabled workers to Cloudflare's edge in the correct mathematical dependency sequence.

Ø=Ý C O N F I G U R A T I O N F I L E S S P E C

The Hoox platform uses a dual configuration file architecture to track workspace states:

A. WRANGLER.JSONC (CENTRAL SETTINGS)

This file represents the declarative single source of truth for your monorepo's active

workers:

```
  {
    "global": {
      "cloudflare_account_id": "debc6545e63bea36be059cbc82d80ec8",
      "subdomain_prefix": "cryptolinx",
    },
    "workers": {
      "d1-worker": {
        "enabled": true,
        "path": "workers/d1-worker",
        "vars": { "database_name": "trade-data-db" },
      },
      "trade-worker": {
        "enabled": true,
        "path": "workers/trade-worker",
        "secrets": ["BYBIT_API_KEY", "BYBIT_API_SECRET", "TELEGRAM_BOT_TOKEN"],
      },
    },
  }
```

B. .INSTALL-WIZARD-STATE.JSON (ONBOARDING STATE)

During the interactive setup, the CLI caches your current step and intermediate inputs inside `.install-wizard-state.json` at your project root.

- **State Recovery**: If your terminal session is disconnected or wrangler login prompts timeout, you can run `hoox init` again. The CLI will detect the state file and seamlessly resume your onboarding from the last incomplete step.
- **Auto-Cleanup**: Upon final completion of Phase 7, the state file is automatically purged to keep your root directory clean.

Ø=Ý SECRET BINDINGS ARCHITECTURE

Hoox utilizes Cloudflare's hardware-secured **Secret Store** to bind environment credentials to V8 isolates without exposing them in git history.

LOCAL MOCKING (`.DEV.VARS`)

During local development, wrangler dev looks for a local, gitignored file called `.dev.vars` inside each worker's directory to simulate secrets:

```
  # workers/trade-worker/.dev.vars
  BYBIT_API_KEY=mock_bybit_development_key
```

```
BYBIT_API_SECRET=mock_bybit_development_secret
```

PRODUCTION SECRET BINDINGS

When deploying to production, wrangler binds these variables using direct encrypted environments in your worker's wrangler configuration:

```
{
  "secrets_store": {
    "bindings": [
      {
        "binding": "BYBIT_API_KEY_BINDING",
        "store_id": "48433bc559a943f09d9d6c622e188fd5",
        "secret_name": "BYBIT_API_KEY"
      }
    ]
  }
}
```

This guarantees that secrets are never logged, never cached in plain text on disk, and are only accessible inside your worker's sandboxed execution isolate memory.

> **Tip:** Made a configuration mistake or changed your subdomain? You can re-run `hoox check-setup` at any time to execute high-integrity type validation and ensure all bindings and configurations match production examples perfectly!

Ø=Ý N E X T S T E P S

- **[DevOps Setup & Operations Manual](setup_and_operations.md)** — Dive into complete operations, variable matrices, and troubleshooting.
- **[Terminal UI Cockpit](tui.md)** — Run, hot-reload, and monitor your local workers via TUI.

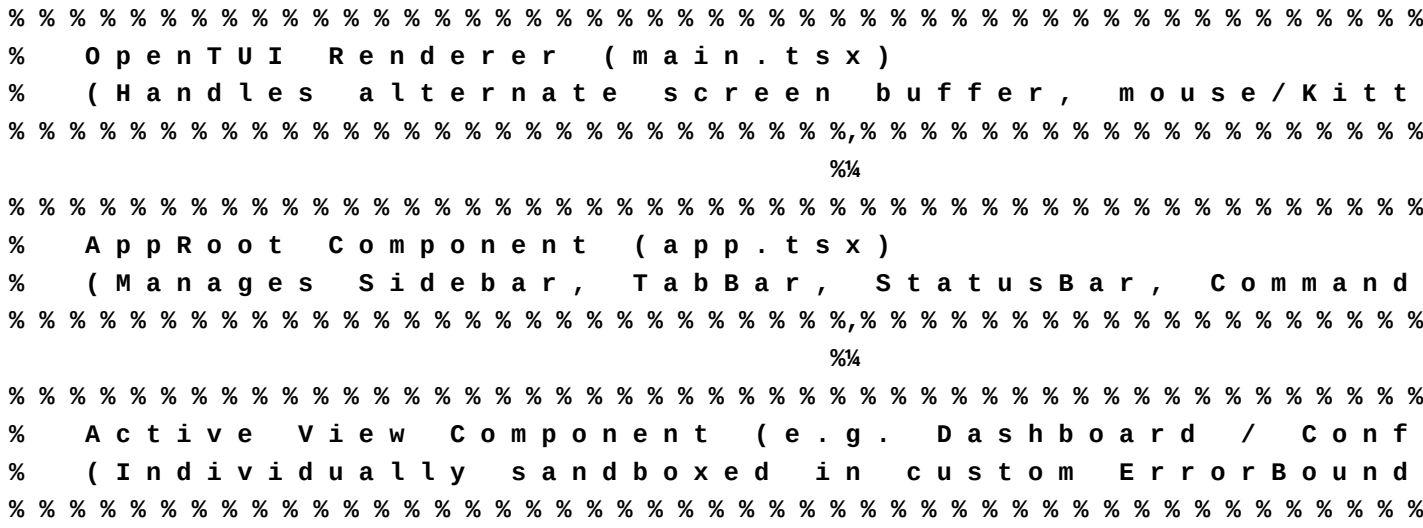
[SECTION: TUI CODE ARCHITECTURE]

Ø=Ý¥þ T U I C O D E A R C H I T E C T U R E

The **Hoox Terminal UI (TUI)** is a full-screen, keyboard-driven operations center built natively using **OpenTUI**, React 19, and Zustand state stores. This document outlines the TUI code architecture, directory structure, data flows, and error recovery mechanisms for senior engineers and system operators.

Ø<ß×þ A R C H I T E C T U R A L B L U E P R I N T

The cockpit is designed as a modular **Single Page Terminal Application (SPTA)**, structured around three decoupled layers:



Ø=ÝÃþ S T O R E A R C H I T E C T U R E & S T A T E F L O W

TUI state is managed using three specialized **Zustand stores** combined with **Immer middleware** to allow safe, mutating updates to deeply nested state trees:

1. UI STORE (`SRC/STORES/UI-STORE.TS`)

- **State Managed**: Active view index, sidebar visibility toggles, active modal overlays (e.g. confirmation dialogs), and the Command Palette fuzzy-search string.
- **Key Actions**: `setView(index)`, `toggleSidebar()`, `pushModal(modal)`, `popModal()`.

2. SERVICE STORE (`SRC/STORES/SERVICE-STORE.TS`)

- **State Managed**: Real-time worker health matrices, trade logs history, analytics

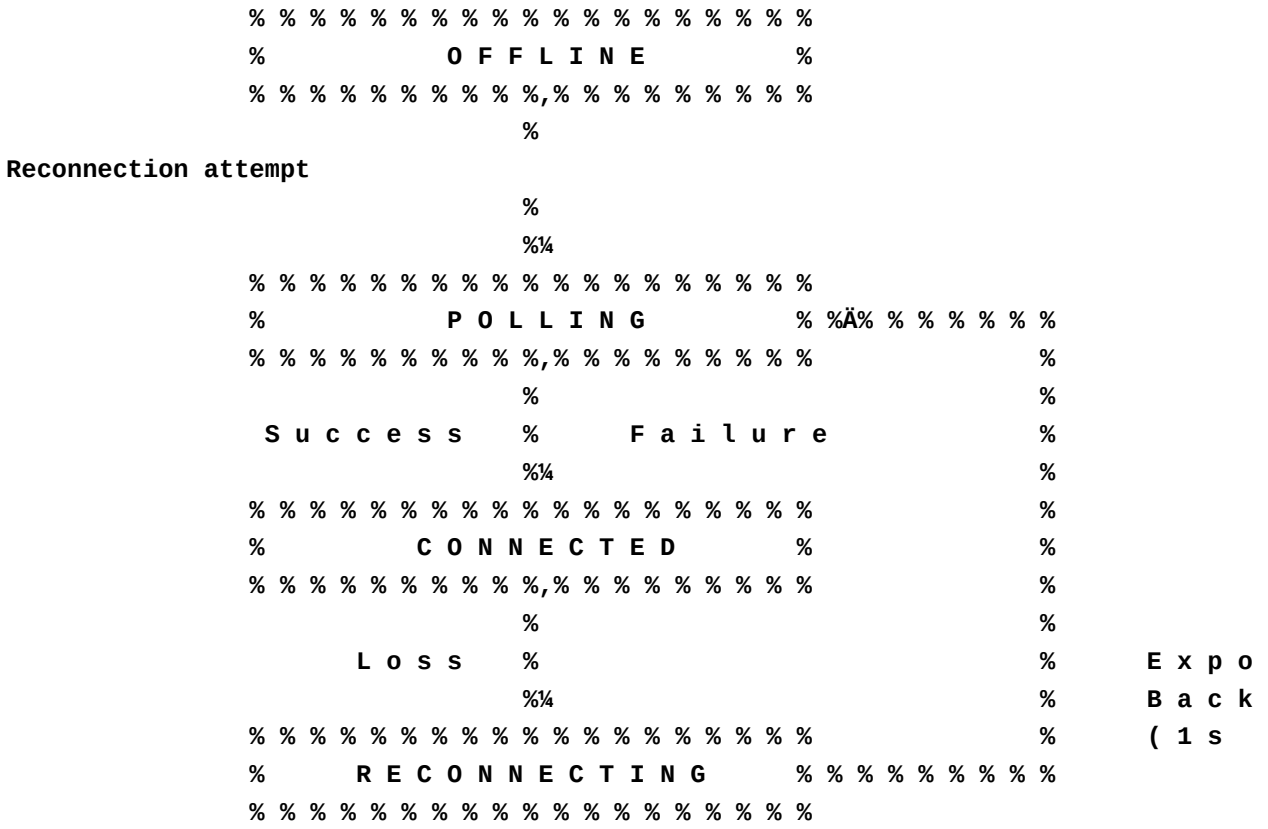
telemetry, log buffers, and the connection status.
- **Key Actions**: `updateWorkerStatus(name, status)`, `addLogEntry(worker, log)`, `addTradeFill(trade)`, `setConnectionState(state)`.

3. CONFIG STORE (`SRC/STORES/CONFIG-STORE.TS`)

- **State Managed**: UI themes (dark/light), data refresh intervals, Telegram alert routes, and custom keyboard binds.
- **Persistence**: Automatically serialized and saved to disk at `~/.hoox/config.json`.

Ø=ÞÜ C O N N E C T I O N R E S I L I E N C E S T A T E M A C H I N E

The TUI maintains a strict connection lifecycle to track connectivity to the local API server or remote worker tunnels:



EXPONENTIAL BACKOFF INTERVALS

If the API connection drops out:

1. The status bar immediately transitions to a yellow/orange pulsing **RECONNECTING** dot.
2. The polling engine schedules reconnect attempts using exponential backoff:
   $$\text{Backoff Interval} = \min(2^{\text{retry_count}} \times 1000\text{ms},

16000\text{ms})$$
3. Active views display a subtle `"Stale Data: Last updated Xm ago"` warning badge, allowing the operator to read static stats without freezing the screen layout.
4. After 5 failed attempts, the state transitions to `OFFLINE` (red dot). The engine continues running minimal background checks and restores live statistics automatically as soon as the API recovers.

Ø=ÜÝ D U A L - C H A N N E L D A T A C O M M U N I C A T I O N

To maintain high performance and low thread overhead:

1. REST API (POLLING CHANNEL)

- **Frequency**: Configurable in `CONFIG_KV` (default: every 2 seconds).
- **Data Transferred**: Core worker health status checks, database statistics, and configuration files.

2. SERVER-SENT EVENTS (REAL-TIME INGESTION CHANNEL)

- **Protocol**: HTTP SSE streaming.
- **Data Transferred**: High-frequency real-time logs and exchange trade fills.
- **Memory Buffer Constraints**: To prevent memory leaks during extended sessions in the terminal, the TUI implements strict ring-buffer limits:
  - **Trade Feed**: 500 entries max.
  - **Log Stream**: 1,000 lines max.
  - **Alert Console**: 100 lines max.
  - _When a buffer limit is hit, the oldest entry is automatically dropped._

Ø=Þáþ D O U B L E - L A Y E R C R A S H P R O T E C T I O N

Since the TUI is designed to be an enterprise-grade command center, a rendering bug or syntax error in one pane must never crash the entire application.

LAYER 1: PER-VIEW ERROR BOUNDARIES

Every view (e.g. `WorkerDetail`, `ConfigEditor`) is wrapped inside a custom React `ErrorBoundary` component:

- If a rendering exception occurs within the `ConfigEditor` (e.g. due to a Tree-sitter WASM parsing exception), the error is caught, a clean error panel is displayed inside that specific tab, and a `[Retry]` button is mounted.
- **No Side Effects**: The rest of the terminal, sidebar, and status bar **continue

running normally**, ensuring uninterrupted monitoring.

LAYER 2: PROCESS-LEVEL TRAP (`CRASHRECOVERYAPP`)

If an uncaught exception or unhandled promise rejection occurs at the root level of the Node/Bun process:

1. The process trap intercepts the signal and prevents a hard exit to a broken shell.
2. Displays a styled Terminal Recovery Screen detailing the error stack.
3. Offers three instant recovery options:
   - **`[Restart]`**: Cleans Zustand stores and performs a fresh reboot.
   - **`[Safe Mode]`**: Disables heavy analytics feeds and WASM syntax highlighting to restore operations in minimal capacity.
   - **`[Report Bug]`**: Automatically exports the error trace and logs to a file at `logs/crash-report.log`.

> **Tip:** Adding custom components or views? Ensure all JSX elements comply with OpenTUI's native terminal components (using only lowercase intrinsic tags like `<box>`, `<text>`, and `<list>`). Absolute layout dimensions must be integers representing terminal characters or `"100%"` values.

Ø=Ý N E X T S T E P S

- **[DevOps Setup & Operations Manual](setup_and_operations.md)** — Complete runbook, variable matrices, and troubleshooting.
- **[Architecture & Edge Topology](architecture/overview.md)** — In-depth architectural outlines and Service Bindings routing maps.

[SECTION: SYSTEM TOPOLOGY & OVERVIEW]

Ø<ß×þ SYSTEM TOPOLOGY & OVERVIEW

Hoox is an enterprise-grade, serverless algorithmic trading platform built entirely on **Cloudflare's Edge V8 isolates** and globally distributed resources. By using a modular, service-oriented architecture, Hoox decomposes complex trading processes into ten highly specialized micro-workers.

These workers communicate privately in microseconds, auto-scale globally near exchange servers, and store transaction logs in localized databases—all while running within Cloudflare's $0 free tiers.

Ø=Ýúþ HIGH-LEVEL SYSTEM ARCHITECTURE

The ecosystem splits public-facing ingress points from private internal compute layers:

```
graph TB
        %% Styling %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    classDef external fill:#f5f5f5,stroke:#999,stroke-width:2,color:#333
    classDef waf fill:#fff8e1,stroke:#f9a825,stroke-width:2
    classDef worker fill:#e8f5e9,stroke:#43a047,stroke-width:2,color:#1b5e20
    classDef storage fill:#e3f2fd,stroke:#1e88e5,stroke-width:2,color:#0d47a1
    classDef compute fill:#f3e5f5,stroke:#8e24aa,stroke-width:2,color:#4a148c
    classDef dash fill:#fff3e0,stroke:#ef6c00,stroke-width:2,color:#bf360c

        %% Ingress Layer %%%%%%%%%%%%%%%%%%%%%%%%
        subgraph Ingress["Ø<ß Public Ingress Layer"
            TV["Ø=ÜÊ TradingView Webhooks"]:::extern
            TG["Ø=Ü¬ Telegram Bot Commands"]:::exter
            EM["Ø=Üç Email Signal Senders"]:::extern
            WAF["Ø>Ýñ Cloudflare WAF / Firewall"]:::
            GW["Ø=Ý hoox Gateway Isolate"]:::worker
    end

        %% Private Compute Layer %%%%%%%%%%%%%%
        subgraph Compute["&¡ Private Internal Edge
            TW["Ø=ÜÈ trade-worker (Execution Engine)
            D1W["Ø=ÝÄþ d1-worker (SQL Hub)"]:::worke
            AW["Ø>Ýà agent-worker (AI Risk Manager)"
            TGW["Ø=Ü¬ telegram-worker (Notifications
            EMW["Ø=Üç email-worker (Email Parser)"]:
            W3W["Ø<ß web3-wallet (DeFi Swaps)"]:::w
            ANW["Ø=ÜÊ analytics-worker (observabilit
            RPW["Ø=ÜÄ report-worker (PDF Generator)"
    end
```

```
        % % % %  S t o r a g e  &  R e s o u r c e  L a y e r % % % % % % % % % % %
        s u b g r a p h  S t o r a g e [ "Ø=Ü¾  P e r s i s t e n t  E d g e  S t o r a
    KV[("KV Config Namespace")]:::storage
    DB[("D1 SQLite Database")]:::storage
    R2[("R2 Logs & PDF Bucket")]:::storage
    VEC[("Vectorize RAG Index")]:::storage
            D O { { "Ø=Ý  D u r a b l e  O b j e c t s  m u t e x " } } : : : c o m
            Q { { "Ø=Üè  Q u e u e s  Q u e u e " } } : : : c o m p u t e
            B R { { "Ø<ß  B r o w s e r  R e n d e r i n g  C h r o m e " } } : : :
end

        % % % %  F l o w  C o n n e c t i o n s % % % % % % % % % % % % % % % % % % % %
TV --> WAF
WAF --> GW
EM --> EMW
TG --> TGW

GW -->|Service Binding| TW
GW -->|Service Binding| TGW
GW -->|Service Binding| ANW
GW -->|Durable Object Lock| DO
GW -->|Queue Failover| Q

TW -->|Service Binding| D1W
TW -->|Service Binding| TGW
TW -->|Service Binding| ANW
TW -->|DeFi Execution| W3W
TW -.->|Config Read| KV
TW -.->|Write Trade Logs| R2

D1W -.->|SQLite queries| DB
TGW -.->|Semantic RAG search| VEC
TGW -->|Service Binding| ANW

AW -->|Cron 5m / Position Scale| TW
AW -->|Service Binding| TGW
AW -->|Service Binding| D1W

RPW -->|Cron 2x/day / Render PDFs| BR
RPW -.->|Save Reports| R2
RPW -->|Push PDF Link| TGW
```

Ø=ÜÊ C O M P R E H E N S I V E M I C R O - W O R K E R C A T A L O G

Worker Name	Runtime Scope	Cron Trigger	Public Routing	Smart Placement	Primary Observability

`hoox`	Gateway Router	No	**Yes** (`/webhook`)	**Yes** (Fast path)	Time-series Telemetry
`trade-worker`	Order Execution	No	No (Isolated)	**Yes** (Exchange Proxied)	Execution Logs
`agent-worker`	Risk Management	Cron `*/5`	No (Isolated)	**Yes** (Account Auditing)	Alert Logs
`telegram-worker`	Alerts & Chat	No	No (Isolated)	**Yes** (Telegram APIs)	Command Logs
`d1-worker`	SQLite Manager	No	No (Isolated)	**Yes** (SQLite Bound)	Query Latency
`report-worker`	Puppeteer PDF	Cron `06,18`	No (Isolated)	**Yes** (Rendering APIs)	Print Status
`email-worker`	IMAP Parsing	Cron `*/5`	No (Isolated)	No	Parse Statistics
`web3-wallet`	DeFi Swap Engine	No	No (Isolated)	No	Tx Sign Logs
`analytics-worker`	Observability	No	No (Isolated)	No	Metrics Dataset

Ø=Þáþ T H E 5 - L A Y E R S E C U R I T Y A R C H I T E C T U R E

Security is designed as concentric protective corridors:

[WAF: IP Range Allow-list] -> [Gateway: Webhook Passkey] -> [Isolation: Service Bindings] -> [Worker Auth: INTERNAL_KEY] -> [Mutex: Durable Objects]

LAYER 1: EDGE-LEVEL FIREWALL & WAF

Cloudflare’s global WAF drop connections immediately at the edge if:

- The payload does not originate from verified TradingView webhook IP ranges.
- The request rate exceeds threshold ceilings (10 requests/minute).

LAYER 2: WEBHOOK PASSKEY AUTHENTICATION

The `hoox` gateway validates that the payload `apiKey` string exactly matches the encrypted `webhooks:api_key` stored inside your `CONFIG_KV` namespace. Mismatched signals are instantly dropped with a `401 Unauthorized` response.

LAYER 3: SERVICE BINDING ENCRYPTED ISOLATION

Internal workers (`trade-worker`, `d1-worker`, `agent-worker`) expose **zero** public HTTP endpoints. They cannot be targeted or accessed from the public internet. They can only be invoked internally by other V8 isolates using Cloudflare **Service Bindings**.

LAYER 4: STANDARDIZED INTERNAL AUTHORIZATION

To prevent internal bypass or privilege escalation, all internal microservice boundaries enforce a strict **bearer authorization check**:

- All internal workers (`hoox`, `trade-worker`, `d1-worker`, `agent-worker`, `telegram-worker`) are bound to the same `INTERNAL_KEY_BINDING` secret.
- Every service-to-service invocation is audited by the shared `requireInternalAuth` middleware from `@jango-blockchained/hoox-shared/middleware`, dropping unauthorized calls.

LAYER 5: DURABLE OBJECT IDEMPOTENCY LOCKS

If the network drops after an order fill, TradingView will resend the webhook. The gateway uses a single-threaded **Durable Object** to lock the request trace ID. If the transaction ID has already been logged, the duplicate is dropped before hitting exchange APIs, preventing double-ordering.

> **Tip:** Smart Placement is enabled across all critical execution paths. This ensures that even though your webhook might hit a Cloudflare edge node in London, the actual transaction logic automatically shifts to Frankfurt or Tokyo (wherever the exchange APIs reside), eliminating network slippage entirely.

Ø=Ý N E X T S T E P S

- **[Worker Communication Specifications](communication.md)** — Deep dive into service bindings, zero-TCP routing, and V8 engines.
- **[Data Flow Maps](data-flow.md)** — Step-by-step sequence charts of trade executions and cron risk evaluations.

[SECTION: ISOLATE COMMUNICATION SPEC]

Ø=Ý I S O L A T E C O M M U N I C A T I O N S P E C

In a traditional server-based monorepo or Docker cluster, microservices communicate over TCP/IP connections using protocols like REST, gRPC, or WebSockets. These introduce significant **networking overhead**: DNS resolution, TCP handshakes, TLS negotiation, and data serialization.

Hoox completely bypasses the networking stack by leveraging Cloudflare's **Service Bindings**. This document details the low-level V8 routing mechanics, internal authentication protocols, and diagnostic mocking configurations of the Hoox communication layer.

&¡ 1 . S E R V I C E B I N D I N G S : Z E R O - O V E R H E A D V

Cloudflare Service Bindings allow one edge worker to call another **without ever hitting the public internet**.

THE UNDER-THE-HOOD V8 MECHANICS

- **Direct Execution**: When `hoox` calls `env.TRADE_SERVICE.fetch()`, the Cloudflare runtime does not construct a TCP packet or route it through a virtual network. Instead, the runtime **spawns the target worker's V8 isolate in the same physical memory thread** and executes its entry point function directly.
- **Zero Serialization Overhead**: Payloads are passed directly as active V8 memory pointers, cutting JSON serialization and parsing costs.
- **Latency Guarantee**: Internal isolate transitions are completed in **under 1 microsecond**, making microservice communication practically instant.

```
// Declarative bindings inside workers/hoox/wrangler.jsonc
{
  "services": [
    { "binding": "TRADE_SERVICE", "service": "trade-worker" },
    { "binding": "TELEGRAM_SERVICE", "service": "telegram-worker" },
    { "binding": "ANALYTICS_SERVICE", "service": "analytics-worker" },
  ],
}
```

Ø=Ü» 2 . C O M P L E T E S E R V I C E I N V O C A T I O N S I M P

Below is the standard, production-grade template used to route authenticated, structured HTTP payloads between workers:

```
import { requireInternalAuth } from "@jango-blockchained/hoox-shared/middleware";

export interface Env {
  TRADE_SERVICE: Fetcher; // Service Binding Fetcher
  INTERNAL_KEY_BINDING: string; // Authorized Internal Key secret
}

export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    // 1. Build and validate payload
    const payload = {
      exchange: "bybit",
      action: "LONG",
      symbol: "BTCUSDT",
      quantity: 0.002,
    };

    // 2. Invoke the internal trade-worker V8 isolate
    try {
      const tradeResponse = await env.TRADE_SERVICE.fetch(
        "https://trade-worker/webhook",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            "X-Internal-Auth-Key": env.INTERNAL_KEY_BINDING, // Bearer Auth
          },
          body: JSON.stringify(payload),
        }
      );

      if (!tradeResponse.ok) {
        throw new Error(
          `Internal binding returned HTTP status: ${tradeResponse.status}`
        );
      }

      const result = await tradeResponse.json();
      return new Response(JSON.stringify({ success: true, result }), {
        status: 200,
        headers: { "Content-Type": "application/json" },
      });
    } catch (error: any) {
      return new Response(
        JSON.stringify({ success: false, error: error.message }),
        {
          status: 502, // Bad Gateway
          headers: { "Content-Type": "application/json" },
        }
      );
```

```
    }
  },
};
```

Ø=ÞÅ 3. INTERNAL AUTHORIZATION MIDDLEWAR

To prevent unauthorized, cross-tenant, or direct internal calls, every internal endpoint is secured using the shared `requireInternalAuth` middleware from `@jango-blockchained/hoox-shared/middleware`:

```
import { requireInternalAuth } from "@jango-blockchained/hoox-shared/middleware";

// Inside target worker's router or fetch handler:
export async function handleRequest(
  request: Request,
  env: Env
): Promise<Response> {
  // Returns 401 Unauthorized Response if the header is invalid or missing
  const authError = requireInternalAuth(request, env, "INTERNAL_KEY_BINDING");
  if (authError) return authError;

  // Key is valid — continue execution
  return new Response("Authorized", { status: 200 });
}
```

> **Standardization Alert:** Every single internal worker (`hoox`, `trade-worker`, `d1-worker`, `agent-worker`, `telegram-worker`, `email-worker`) binds the exact same secret name: `INTERNAL_KEY_BINDING`. This eliminates variable footprint drift and simplifies secret deployments across your workspace.

Ø>Ýê 4. TESTING & MOCKING SERVICE BINDIN

During local testing (via native `bun test`), you can mock the Service Binding `Fetcher` object cleanly to run full-coverage unit tests without provisioning real Cloudflare APIs:

```
import { expect, test, mock } from "bun:test";

test("Should mock internal trade-worker service bindings", async () => {
  const mockEnv = {
    INTERNAL_KEY_BINDING: "secret_local_test_key",
    TRADE_SERVICE: {
      fetch: async (url: string, init?: RequestInit) => {
        // Confirm headers are present and valid
        const headers = init?.headers as Record<string, string>;
        if (headers["X-Internal-Auth-Key"] !== "secret_local_test_key") {
```

```
            return new Response(JSON.stringify({ success: false }), {
              status: 401,
            });
          }

          return new Response(
            JSON.stringify({
              success: true,
              orderId: "mock_order_10482",
            }),
            { status: 200 }
          );
        },
      } as Fetcher,
    };

    // Run call test assertions
    const res = await mockEnv.TRADE_SERVICE.fetch(
      "https://trade-worker/webhook",
      {
        headers: { "X-Internal-Auth-Key": "secret_local_test_key" },
      }
    );

    const data = await res.json();
    expect(res.status).toBe(200);
    expect(data.orderId).toBe("mock_order_10482");
  });
```

Ø=Ý N E X T S T E P S

- **[Data Flow Mapping](data-flow.md)** — Step-by-step sequence charts of trade executions, backups, and metrics.
- **[Bindings Catalog](bindings.md)** — Review complete environment namespaces and bound objects.

[SECTION: SYSTEM DATA ROUTING SPEC]

Ø<ß S Y S T E M D A T A R O U T I N G S P E C

Hoox operates as a highly orchestrated **distributed event loop**. Because execution logic is split into isolated compute nodes, data flows recursively through multiple V8 transitions, asynchronous queues, time-series datasets, and database ledgers.

This document provides complete, low-level technical specifications and Mermaid sequence diagrams for our four primary data routing pipelines: Webhook Trade execution, AI Risk management, PDF browser rendering, and Observability tracking.

---

1. WEBHOOK TO TRADE EXECUTION FLOW (HIGH-SPEED PATH)

This is the primary transaction pipeline. When a trade signal is received, the system validates the payload, locks the trace ID, executes the order at the edge closest to the exchange, records the fill, and alerts the user.

```
sequenceDiagram
    autonumber
    actor TV as Signal Source (TradingView)
    participant GW as hoox Gateway
    participant DO as IdempotencyStore (Durable Object)
    participant TW as trade-worker
    participant D1 as d1-worker
    participant TG as telegram-worker
    participant AE as analytics-worker

    TV->>GW: POST /webhook (Ingest raw JSON signal)
    Note over GW: Extract Trace ID & check KV Rate Limiter
    GW->>DO: lockTransaction(traceId) (SQLite atomic mutex check)
    alt Mutex Lock Denied (Trace ID Already Logged)
        DO-->>GW: Fail (Conflict: Duplicate Request)
        GW-->>TV: 409 Conflict (Duplicate Request)
    else Mutex Lock Granted (Unique Request)
        DO->>GW: Lock Acquired (Success)
        GW->>TW: Service Binding invoke: POST /webhook (X-Internal-Auth-Key)
        Note over TW: Fetch encrypted Secrets & calculate HMAC-SHA256 signature
        TW->>D1: Service Binding invoke: POST /query (Write "TRADE_PENDING" row)
        TW->>TW: Send Order to Centralized Exchange API
        Note over TW: Exchange fills order & returns status details
        TW->>D1: Service Binding invoke: POST /query (Update Status to "Filled")
        TW->>TG: Service Binding invoke: POST /alert (Format alert template)
        TG-->>TV: Dispatch Telegram Push Alert to User's phone
        TW-->>GW: Return Executed Order JSON
        GW->>AE: Service Binding invoke: POST /track (Log latency & status metrics)
        GW-->>TV: 200 OK (Filled order metadata)
```

```
    end
```

---

2. AUTONOMOUS AI RISK MONITORING FLOW (CRON CYCLE)

Running on a strict **5-minute Cron schedule**, the risk management loop queries SQLite records, audits active exposures, calculates trailing stop deviations, and manages emergency halts.

```
  sequenceDiagram
      autonumber
      participant Cron as Cloudflare Cron Trigger
      participant AW as agent-worker (AI Risk Manager)
      participant D1 as d1-worker (SQL Hub)
      participant TW as trade-worker (Execution)
      participant TG as telegram-worker (Alerts)

      Cron->>AW: Fire trigger: */5 * * * *
      AW->>D1: Service Binding invoke: POST /query (Get open margin positions)
      D1-->>AW: Return position matrices (Average price, leverage, unrealized P&L)
      Note over AW: Evaluate Max Daily Drawdown threshold against balance
      alt Drawdown Limit Hit (USDT Daily loss > max_daily_drawdown_percent)
                  Note over AW: EMERGENCY HALT TRIGGERED
          AW->>AW: Set trade:kill_switch = true in CONFIG_KV
          AW->>TW: Service Binding invoke: POST /close-all (Flatten all active exposure)
          TW-->>AW: All positions flattened successfully
          AW->>TG: Service Binding invoke: POST /alert (Dispatch Emergency Alert)
          TG-->>AW: User notified on phone
      else Normal Operation (Within Safe Margins)
          Note over AW: Calculate Trailing Stop-Loss price deviations
          alt Deviation Triggered (Price crossed dynamic stop-loss boundary)
              AW->>TW: Service Binding invoke: POST /order (Close target position)
              TW-->>AW: Order filled
              AW->>TG: Service Binding invoke: POST /alert (Stop-Loss Triggered
Notification)
          end
      end
```

---

3. PDF PORTFOLIO REPORT RENDERING FLOW

Runs twice daily to automate HTML dashboard rendering, compile PDFs via Puppeteer on the edge, offload to R2 storage, and dispatch download corridors.

```
  sequenceDiagram
      autonumber
      participant Cron as Cloudflare Cron Trigger
```

```
    participant RP as report-worker
    participant BR as Browser Rendering API (Chrome Isolate)
    participant R2 as R2 Object Storage Bucket
    participant TG as telegram-worker

    Cron->>RP: Fire trigger: 06:00 & 18:00 UTC
    RP->>RP: Query D1 P&L history & compile HTML5 template
    RP->>BR: POST /browser-rendering/pdf (HTML string payload)
    Note over BR: Spin up headless Puppeteer Chrome & render viewport
    BR-->>RP: Return compiled PDF Buffer stream
    RP->>R2: PutObject: reports/daily-report-{timestamp}.pdf
    RP->>TG: Service Binding invoke: POST /alert (P&L report link)
    TG-->>RP: Dispatch report download link to user
```

---

4. OBSERVABILITY & TIME-SERIES ANALYTICS FLOW

To maintain complete cross-worker telemetry without blocking critical order threads, Hoox routes analytics data points asynchronously to a dedicated metrics warehouse.

```
sequenceDiagram
    autonumber
    participant W as Any Compute Worker (hoox, trade, agent, etc.)
    participant AN as analytics-worker (SQL Analytics Ingest)
    participant AE as Cloudflare Analytics Engine

    W->>AN: Service Binding invoke: POST /track/api-call (Trace ID, latencyMs, success)
    Note over AN: Sanitize metrics inputs & format structural time-series logs
    AN->>AE: writeDataPoint({ blobs: [worker, endpoint], doubles: [latencyMs] })
    Note over AE: Persistent logging in time-series warehouse for Next.js dashboard
```

Ø=Ü¾ 5. GLOBAL DATA PERSISTENCE MAPPING

Storage Platform	Namespace / Database Name	Data Payload Details	Associated Compute Workers
D1 Database	`trade-data-db` (SQLite)	Executed fills, open position matrices, Drizzle tracking logs.	`d1-worker`, `trade-worker`, `agent-worker`
CONFIG_KV	`CONFIG_KV` (Key-Value)	16-key global runtime manifest, emergency Kill Switch.	**All Workers** + Next.js Dashboard
SESSIONS_KV	`SESSIONS_KV` (Key-Value)	Session access states and API authorization cookies.	`hoox` Gateway
R2 Storage	`trade-reports` (S3 Bucket)	Compiled PDF portfolio reports.	

`report-worker`
R2 Storage	`hoox-system-logs` (S3 Bucket)	Verbose JSON exchange API payloads (REST & WebSocket logs).	`trade-worker`
Vectorize	`my-rag-index` (Vector DB)	Semantic chat and history vector embeddings.	`telegram-worker`

Ø=Ý N E X T S T E P S

- **[Bindings Catalog](bindings.md)** — Check wrangler settings and resource declarations.
- **[Storage Engineering Manual](storage.md)** — Dive into Drizzle database schemas and SQLite properties.

[SECTION: INFRASTRUCTURE BINDINGS INDEX]

Ø>Ýì I N F R A S T R U C T U R E B I N D I N G S I N D E X

In Cloudflare’s serverless architecture, **bindings** represent the declarative bridges linking your isolate compute logic to other internal microservices and storage platforms. This document serves as the absolute, production-grade reference registry for all resource bindings configured in the Hoox monorepo.

---

1. SECRETS & ENVIRONMENT VARIABLES MATRIX

These parameters represent encrypted variables injected directly into V8 execution isolates at runtime.

Variable Name	Type	Bound Workers	Operational Impact
`INTERNAL_KEY_BINDING`	Secret	`hoox`, `trade-worker`, `d1-worker`, `telegram-worker`, `email-worker`	Cryptographic auth key used by the `requireInternalAuth` middleware to validate service-to-service calls.
`TG_BOT_TOKEN_BINDING`	Secret	`telegram-worker`	Secret bot token issued by `@BotFather` to authenticate Telegram API commands and alerts.
`TELEGRAM_SECRET_TOKEN`	Secret	`telegram-worker`	Webhook verification token to authorize Telegram push events.
`WEBHOOK_API_KEY`	Secret	`hoox`	General passkey validated during incoming webhook signal requests.
`BYBIT_API_KEY` / `_SECRET`	Secret	`trade-worker`	Credentials used to execute cryptographically signed order routes to Bybit's APIs.
`BINANCE_API_KEY` / `_SECRET`	Secret	`trade-worker`	Credentials used to execute trade routes to Binance's APIs.
`MEXC_API_KEY` / `_SECRET`	Secret	`trade-worker`	Credentials used to execute trade routes to MEXC's APIs.
`CF_API_TOKEN`	Secret	`report-worker`	Access token used to invoke the Cloudflare Puppeteer Browser Rendering APIs.

---

2. SERVICE BINDINGS (COMPUTE CONNECTORS)

Service Bindings link workers' V8 runtimes together locally in memory, with microsecond latency and zero external internet hops.

Binding Name	Ingesting Worker	Target Service	Purpose
`TRADE_SERVICE`	`hoox`, `agent-worker`, `email-worker`	`trade-worker`	Handles leverage calculation, size scaling, and execution.
`TELEGRAM_SERVICE`	`hoox`, `trade-worker`, `agent-worker`, `report-worker`	`telegram-worker`	Dispatches real-time alerts and parses slash commands.
`D1_SERVICE`	`trade-worker`, `agent-worker`, `report-worker`, `dashboard`	`d1-worker`	Serves as the high-integrity proxy SQL data manager.
`AGENT_SERVICE`	`dashboard`	`agent-worker`	Triggers risk audits, chat streams, and telemetry updates.

---

3. KV NAMESPACE CACHES

Key-Value caches store parameters that require sub-millisecond read access.

Binding Name	Bound Workers	Purpose
`CONFIG_KV`	**All Workers** + Dashboard	Primary cache for the 16-key runtime manifest, rate-limiter, and Kill Switch.
`SESSIONS_KV`	`hoox` Gateway	Stores active webhook session credentials and token cookie states.

---

4. ASYNCHRONOUS QUEUES

Queues guarantee message delivery during times of heavy exchange network congestion or API rate limits.

Binding Name	Ingesting Worker	Queue Name	Type	Operational Action

:-- |
| `TRADE_QUEUE` | `hoox` Gateway | `trade-execution` | **Producer** | Serializes and enqueues signal payloads during failover. |
| `TRADE_QUEUE` | `trade-worker` | `trade-execution` | **Consumer** | Pulls enqueued signals and retries executions with backoff. |

---

5. SQLITE-BACKED DURABLE OBJECTS

Durable Objects enforce exactly-once execution, preventing catastrophic double-trading.

Binding Name	Bound Worker	Target Class Name	Purpose
`IDEMPOTENCY_STORE`	`hoox` Gateway	`IdempotencyStore`	Mutex locking engine with local SQLite and auto-alarm garbage collection.

---

6. R2 OBJECT STORAGE BUCKETS

R2 Buckets store heavy files with zero bandwidth egress retrieval fees.

Binding Name	Bound Workers	Bucket Name	Target Asset Payload
`REPORTS_BUCKET`	`trade-worker`, `report-worker`	`trade-reports`	Compiled PDF daily/weekly portfolio reports.
`SYSTEM_LOGS_BUCKET`	`trade-worker`	`hoox-system-logs`	Verbose exchange API request-response logs.
`UPLOADS_BUCKET`	`telegram-worker`	`user-uploads`	User chart screenshots and conversation images.

---

7. D1 SQLITE DATABASES

Binding Name	Bound Workers	Database Instance Name	Purpose
`DB`	`d1-worker`	`trade-data-db`	Primary transactional trade database storage.

---

8. WORKERS AI & VECTORIZE INDEXES

Binding Name	Bound Workers	Target Asset Name	Operational Purpose
`AI`	`hoox`, `trade-worker`, `agent-worker`, `telegram-worker`	Edge LLM Models	Runs LLaMA-3 inference, risk analysis, and chat.
`VECTORIZE_INDEX`	`telegram-worker`	`rag-index`	Custom semantic search vector DB for RAG queries.

Ø<ß 9. P U P P E T E E R B R O W S E R R E N D E R I N G

The `report-worker` invokes Cloudflare’s Browser Rendering Chrome isolates using a secure **REST API** (no binding required):

- **Route**: `POST https://api.cloudflare.com/client/v4/accounts/{account_id}/browser-rendering/pdf`
- **Headers**:
  ```http
  Authorization: Bearer <CF_API_TOKEN_BINDING>
  Content-Type: application/json
  ```
- **JSON Payload**:
  ```json
  {
    "html": "<html>...</html>",
    "options": {
      "format": "A4",
      "printBackground": true
    }
  }
  ```

> **Tip:** Adding new bindings to your workers? Always update your `wrangler.jsonc` manifest at the workspace root, and then execute `hoox deploy update-internal-urls` to sync bindings and URLs globally!

Ø=Ý N E X T S T E P S

- **[Storage & SQLite DDL](storage.md)** — Dive into Drizzle schemas, R2 bucket configurations, and database rules.
- **[Production Deployments](../deployment/production.md)** — Learn how Wrangler compiles and maps these bindings to the live edge.

[SECTION: STORAGE & DATA ENGINEERING SPEC]

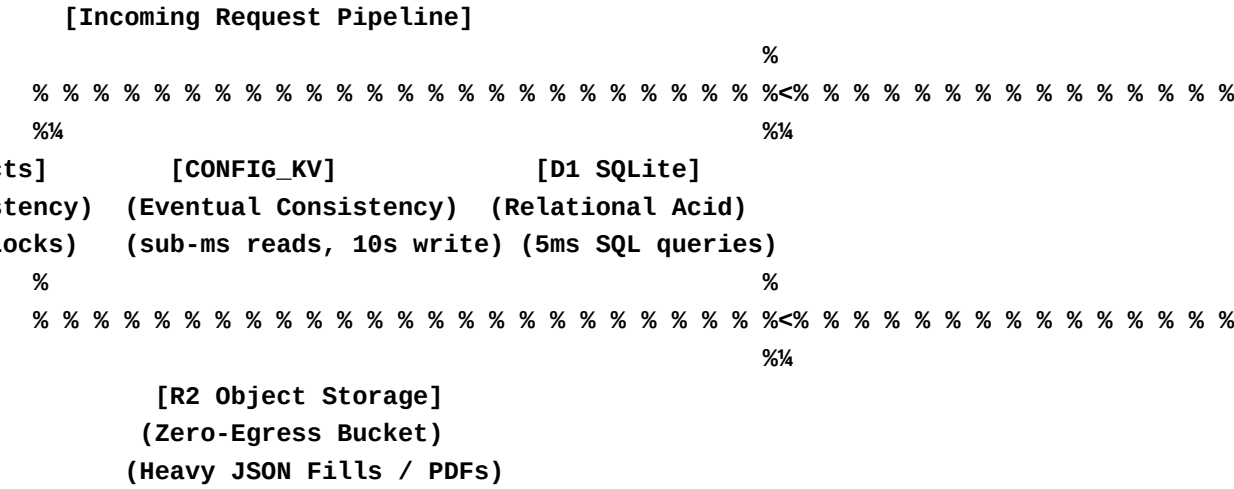
Ø=Ü¾ S T O R A G E & D A T A E N G I N E E R I N G S P E

Hoox operates a **multi-tier edge storage topology** to balance high-speed read/write performance, relational transactional safety, and long-term analytical capacity. By separating high-frequency dynamic states from static parameters and verbose event logs, Hoox ensures that latency remains in the single-digit milliseconds while keeping storage costs at $0/month.

---

1. THE MULTI-TIER STORAGE ARCHITECTURE

Hoox segregates data into four distinct edge storage tiers based on consistency, read/write latency, and size constraints:



Storage primitive	Engine Technology	Consistency Model	Read Latency	Write Latency	Ideal Use Case
Durable Objects	In-Memory Mutex	Strong Consistency	**< 2ms**	**< 5ms**	Atomic transaction locks, high-frequency concurrency dedup.
Workers KV	Distributed Cache	Eventual Consistency	**< 1ms**	**< 10s**	Dynamic exchange routing paths, emergency Kill Switch.
D1 Database	Edge SQLite Isolate	Relational ACID	**< 5ms**	**< 12ms**	Fills ledger, open positions matrix, margin balance records.
R2 Storage	S3 Object Bucket	Strong (per-object)	**< 35ms**	**< 50ms**	Verbose exchange API request/response JSONs, PDF reports.

Ø=ÝÄþ 2 . D 1 R E L A T I O N A L E N G I N E & D R I Z Z L E

D1 runs a serverless SQLite engine embedded in the Cloudflare V8 worker isolate thread. This eliminates TCP handshake overhead when executing SQL queries.

DRIZZLE ORM SCHEMA STANDARDS

Hoox developers use Drizzle ORM to define strict type safety for D1 SQLite tables. Below is the active specification for our `trades` and `positions` schemas:

```
import { sqliteTable, text, real, integer } from "drizzle-orm/sqlite-core";

// 1. Transactional Trades Ledger Schema
export const trades = sqliteTable("trades", {
  id: text("id").primaryKey(), // UUIDv4
  requestId: text("request_id").notNull(), // Distributed Trace ID
  exchange: text("exchange").notNull(), // "bybit" | "binance" | "mexc"
  symbol: text("symbol").notNull(), // Uppercase symbol: "BTCUSDT"
  action: text("action").notNull(), // "LONG" | "SHORT" | "CLOSE"
  side: text("side").notNull(), // "BUY" | "SELL"
  quantity: real("quantity").notNull(), // Executed contract quantity
  price: real("price").notNull(), // Execution fill price
  fee: real("fee").notNull(), // Exchange transaction fee paid in quote
  orderId: text("order_id").notNull(), // Exchange-provided order identifier
  status: text("status").notNull(), // "Filled" | "Failed"
  createdAt: integer("created_at", { mode: "timestamp" }).default(new Date()),
});

// 2. Real-Time Open Position Matrix Schema
export const positions = sqliteTable("positions", {
  symbol: text("symbol").primaryKey(),
  exchange: text("exchange").notNull(),
  side: text("side").notNull(), // "LONG" | "SHORT"
  size: real("size").notNull(), // Current contract size
  entryPrice: real("entry_price").notNull(), // Volume-weighted average entry price
  leverage: integer("leverage").default(1),
  updatedAt: integer("updated_at", { mode: "timestamp" }).default(new Date()),
});
```

Ø=Üæ 3 . R 2 L O G O F F L O A D I N G : S Q L I T E C O N S E R

SQLite engines are single-writer databases. If write concurrency is too high, transaction queues can lock the thread. To conserve D1 write limits and maintain high performance during high-frequency events:

1. **D1 Relational Logs**: Only high-value financial transactions (the structured fields in the `trades` table above) are written to D1.
2. **R2 Verbose Blobs**: Full, verbose JSON payload exchanges, network headers, and

WebSocket stream records are serialized and saved to **R2 Storage** using date-based key paths:
`logs/bybit/BTCUSDT/2026-05-19/order-18049284739.json`

This offloading strategy reduces D1 SQLite write volumes by **up to 75%**, keeping your database compact, fast, and fully within free-tier limits.

Ø=Ý 4. KV EVENTUAL CONSISTENCY & CACHE

Cloudflare KV is a highly distributed key-value store optimized for high-read/low-write operations:

- **Eventual Consistency**: When you toggle a setting in KV (e.g. `trade:kill_switch = true`), the change is instantly cached at your local gateway node. However, it takes up to **10 seconds** to propagate to Cloudflare’s other 330+ locations globally.
- **Operational Rule**: KV is perfect for global configurations, allowlists, and emergency kill switches. It is **never** used to track highly dynamic real-time states like position sizes, account drawdowns, or margin balances. High-frequency states must always be routed through D1 or Durable Objects.

> **Danger:** Never attempt to run raw migration files directly on your production D1 database without first running a backup. Ensure your database status is fully tracked and synchronized using Drizzle migrations: `hoox db migrate --remote`.

Ø=Ý NEXT STEPS

- **[Internal Endpoints Mapping](endpoints.md)** — Map out exposed ports, URLs, and service bindings.
- **[Wrangler Setup & Tooling](../development/local-dev.md)** — Configure Wrangler to bind local D1 and KV instances for dev testing.

[SECTION: INTERNAL ENDPOINTS MAP]

Ø=Ý I N T E R N A L E N D P O I N T S M A P

This document catalogs all internal REST, service-to-service, and queue endpoints exposed across the Hoox microservice monorepo. Because internal workers have zero public IP footprints and are completely isolated by Cloudflare’s Zero Trust service bindings, this map serves as the primary integration blueprint for routing, debugging, and dashboard interactions.

Ø<ß×þ I N T E R A C T I V E C O M P U T E & R O U T I N G F L O W

All external webhooks flow through the public `hoox` gateway, which authenticates payloads and routes them to private workers inside localized V8 engine isolates:

```
graph TD
    %% Ingress
    Client[TradingView / User] -->|1. Public HTTPS Ingress| GW[hoox Gateway]

    %% Gateway Service Bindings
    GW -->|2. Service Binding| TW[trade-worker]
    GW -->|2. Service Binding| TGW[telegram-worker]
    GW -->|2. Async Failover Queue| Q[trade-execution Queue]

    %% Internal Workers Services
    TW -->|3. Service Binding| D1W[d1-worker]
    TW -->|3. Service Binding| W3[web3-wallet-worker]
    TGW -.->|RAG Queries| VEC[(Vectorize Index)]
    Q -->|4. Async Consumer| TW
    D1W -.->|SQLite read/write| DB[(D1 Database)]
```

Ø=ÝÂþ E N D P O I N T S D I R E C T O R Y B Y W O R K E R

Every internal HTTP request between V8 isolates must transmit the standard bearer header: `X-Internal-Auth-Key: <INTERNAL_KEY_BINDING>`

Ø=Ý 1 . ` H O O X ` (G A T E W A Y R O U T E R)

- **Status**: Public Ingress Node
- **Bindings Mounts**: `TRADE_SERVICE`, `TELEGRAM_SERVICE`, `ANALYTICS_SERVICE`, `CONFIG_KV`, `TRADE_QUEUE`

Route	Method	Description	Request Shape	Success Response
`/webhook`	`POST`	Primary webhook receiver for TradingView alerts.	`WebhookSignal` JSON	`200 OK` (Orderfilled metadata)
`/health`	`GET`	Probes gateway, D1 connectivity, and DO status.	N/A	`{"status": "ok"}`
`/telegram-webhook`	`POST`	Processes chat commands pushed from Telegram.	Telegram Updates	`200 OK`

Ø=ÜÈ 2. `TRADE-WORKER` (EXECUTION ENGINE)

- **Status**: Private Compute Node (No Public URL)
- **Bindings Mounts**: `D1_SERVICE`, `TELEGRAM_SERVICE`, `ANALYTICS_SERVICE`, `CONFIG_KV`

Route	Method	Description	Request Shape	Success Response
`/webhook`	`POST`	Direct fast-path execution trigger.	`WebhookSignal` JSON	`200 OK` (Fill detail JSON)
`/dex`	`POST`	Dispatches EVM orders on-chain via web3 wallet.	DexTrade JSON	`200 OK` (Tx Hash metadata)
`/api/signals`	`GET`	Retrieves recent signal logs from D1.	Query params filters	`200 OK` (Array of signals)
`/health`	`GET`	Probes CPU thread state and exchange connections.	N/A	`{"status": "ok"}`

Ø=ÝÄþ 3. `D1-WORKER` (SQLITE HUB)

- **Status**: Private Data Proxy (No Public URL)
- **Bindings Mounts**: `DB` (D1 SQLite database binding)

Route	Method	Description	Request Shape	Success Response

`/query`	`POST`	Executes a single SQL query against the SQLite database.	`{"query": "SELECT * FROM trades", "params": []}`	`{"success": true, "results": [...]}`
`/batch`	`POST`	Executes multiple transactional SQL operations.	`{"queries": [{"query": "...", "params": []}]}`	`{"success": true, "results": [...]}`
`/api/dashboard/stats`	`GET`	Computes aggregated Win Rate, drawdown, and daily totals.	N/A	`{"success": true, "stats": {...}}`
`/{tableName}`	`GET`	Lists rows inside a specific SQLite table (with filters).	Query params	`{"success": true, "rows": [...]}`

Ø>Ýà 4. `AGENT-WORKER` (AI RISK MANAGER)

- **Status**: Private Compute Node (Runs primarily on Cron schedule `*/5 * * * *`)
- **Bindings Mounts**: `TRADE_SERVICE`, `D1_SERVICE`, `TELEGRAM_SERVICE`, `AI`

Route	Method	Description	Request Shape	Success Response
`/agent/chat`	`POST`	Starts a conversational market/risk query (SSE supported).	`{"prompt": "...", "stream": true}`	`text/event-stream` stream
`/agent/vision`	`POST`	Analyzes image bytes using multimodal AI models.	`{"image": "base64...", "prompt": "..."}`	`{"analysis": "..."}`
`/agent/status`	`GET`	Returns active trailing stops and current drawdowns.	N/A	`{"status": "active", "stops": []}`
`/health`	`GET`	Probes AI model availability and Cron loop timers.	N/A	`{"status": "ok"}`

Ø=Ü¬ 5. `TELEGRAM-WORKER` (PUSH ALERTS)

- **Status**: Private Compute Node
- **Bindings Mounts**: `ANALYTICS_SERVICE`, `AI`, `VECTORIZE_INDEX`

Route	Method	Description	Request Shape	Success Response
`/alert`	`POST`	Sends a push trade fill notification or daily digest.	`{"chatId": "...", "message": "..."}`	`{"success": true}`

| `/health` | `GET` | Probes Telegram API connection. | N/A | `{"status": "ok"}` |

> **Tip:** Every internal-to-internal transaction automatically inherits the `requestId` trace UUID generated by the gateway. This trace ID is attached as the `X-Request-Id` header, allowing you to trace a single webhook alert across D1 database writes, R2 log outputs, and Telegram alerts instantly!

Ø=Ý N E X T S T E P S

- **[Astro Docs Site Config](../getting-started/configuration.md)** — Map out your build-time environment configurations.
- **[System Storage Architecture](storage.md)** — Deep dive into R2 logs offloading and Drizzle ORM schemas.

[SECTION: VISUAL TOKENS & DESIGN SYSTEM]

Ø<ß¨ V I S U A L T O K E N S & D E S I G N S Y S T E M

To maintain visual cohesion across the entire Hoox ecosystem (web landing pages, Astro documentation sites, Zustand Next.js dashboards, and terminal UIs), the platform implements a standardized, high-integrity design system.

This document catalogs our color tokens, spacing densities, border-radius behaviors, and provides the complete **SVG Icon Mapping Catalog** used inside the documentation navigation chrome.

Ø<ß¨ 1 . O K L C H C O L O R P A L E T T E

Hoox utilizes OKLCH color values to ensure perceptually uniform brightness and high contrast across all edge screens and operating systems:

DARK MODE (PRIMARY VISUAL STANDARD)

- **Background (`--background`)**: `oklch(0.08 0 0)` — Deep charcoal/black.
- **Foreground (`--foreground`)**: `oklch(0.95 0 0)` — Bright neutral Zinc/white.
- **Card (`--card`)**: `oklch(0.12 0 0)` — Slightly elevated dark panel grey.
- **Accent (`--accent`)**: `oklch(0.70 0.20 45)` — High-contrast bright orange (used for focal actions and headers).
- **Muted Foreground (`--muted-foreground`)**: `oklch(0.68 0 0)` — Secondary readability text.
- **Borders (`--border`)**: `oklch(0.30 0 0)` — Low-contrast dark grey dividers.

Ø=ÜÐ 2 . L A Y O U T , B O R D E R S & D E N S I T Y

- **Border Radius (`--radius`)**: Restored globally to `0.5rem` (8px) to soften panel boundaries without losing the clean command-center aesthetic.
- **Shadows**: Cards and active modal prompts bind a highly contrasting deep shadow:
  `shadow-2xl` -> `box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);`
- **Spacing Density**: Standardize grid gutters using Tailwind flex/grid spaces:
  - Cards Grid: `gap-6` (24px) for dashboard cards.
  - Sidebar Items: `gap-0.5` (2px) vertical padding between nav links.

Ø=Ý$ 3 . T Y P O G R A P H Y & R U N T I M E S

Aesthetic Layer	Font Family	CSS Utility	Ideal Operational Use Case
Primary Titles	`"Bebas Neue", sans-serif`	`font-heading`	Main high-signal card headers, page titles.
Prose & Body	`"IBM Plex Sans Variable", sans-serif`	`font-sans`	Explanatory text, paragraphs, tables.
Technical Chrome	`"IBM Plex Mono", monospace`	`font-mono`	Navigation links, settings inputs, SQL queries, code.

Ø=Üæ 4. SVG ICON MAPPING CATALOG

To ensure visual consistency and completely eliminate platform-inconsistent emojis, the navigation chrome (`Sidebar.astro` and `MobileNav.astro`) maps dynamic section keys to clean, flat, inline SVGs:

A. ROCKET ICON (`GETTING-STARTED`)

Used to represent system setup and installation paths.

```
  <svg
    class="size-3.5"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
  >
    <path
      d="M4.5 16.5c-1.5 1.26-2 5-2 5s3.74-.5 5-2c.71-.84.7-2.13-.09-2.91a2.18 2.18 0 0 0-2.91-.09z"
    />
    <path
      d="M12 15l-3-3a22 22 0 0 1 2-3.95A12.88 12.88 0 0 1 22 2c0 2.72-.78 7.5-6 11a22.35 22.35 0 0 1-4 2z"
    />
    <path d="M9 12H4s.55-3.03 2-4c1.62-1.08 5 0 5 0" />
    <path d="M12 15v5s3.03-.55 4-2c1.08-1.62 0-5 0-5" />
  </svg>
```

B. BOOK ICON (`GUIDES`)

Used to represent operational manuals and setup instructions.

```
  <svg
    class="size-3.5"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
  >
    <path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20" />
    <path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z" />
  </svg>
```

C. LIGHTBULB ICON (`CONCEPTS`)

Used to represent structural theories and edge architectures.

```
  <svg
    class="size-3.5"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
  >
    <path d="M9 18h6" />
    <path d="M10 22h4" />
    <path
      d="M15.09 14c.18-.98.65-1.74 1.41-2.5A4.65 4.65 0 0 0 18 8 6 6 0 0 0 6 8c0 1 .23
2.23 1.5 3.5A4.61 4.61 0 0 1 8.91 14"
    />
  </svg>
```

D. BOOK-OPEN ICON (`REFERENCE`)

Used to represent dictionary definitions, configuration matrices, and API details.

```
  <svg
    class="size-3.5"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
  >
    <path d="M2 3h6a4 4 0 0 1 4 4v14a3 3 0 0 0-3-3H2z" />
    <path d="M22 3h-6a4 4 0 0 0-4 4v14a3 3 0 0 1 3-3h7z" />
  </svg>
```

E. TARGET ICON (`TUTORIALS`)

Used to represent step-by-step walkthroughs and integration guides.

```
  <svg
    class="size-3.5"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
  >
    <circle cx="12" cy="12" r="10" />
    <circle cx="12" cy="12" r="6" />
    <circle cx="12" cy="12" r="2" />
  </svg>
```

F. GEAR ICON (`DEVOPS` & `WORKERS`)

Used to represent background cron engines, system setups, and deployment configs.

```
  <svg
    class="size-3.5"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
  >
    <circle cx="12" cy="12" r="3" />
    <path
      d="M12 1v2M12 21v2M4.22 4.22l1.42 1.42M18.36 18.36l1.42 1.42M1 12h2M21 12h2M4.22
19.78l1.42-1.42M18.36 5.64l1.42-1.42"
    />
  </svg>
```

G. HOME HOUSE ICON (`HOME`)

Used to return to the parent portal.

```
  <svg
    class="size-4"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
```

```
    stroke-width="2"
  >
    <path d="M3 9l9-7 9 7v11a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2z" />
    <polyline points="9 22 9 12 15 12 15 22" />
  </svg>
```

Ø=Ý N E X T S T E P S

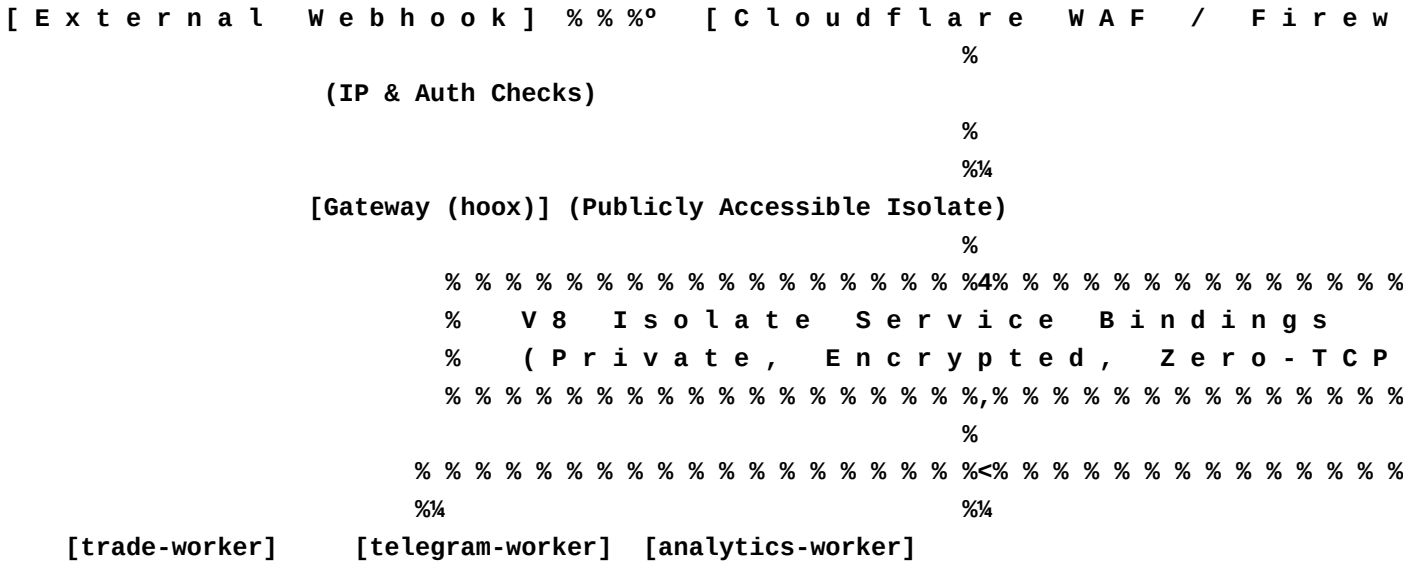
- **[System Topology Overview](overview.md)** — Analyze edge isolates and data layers integrations.
- **[Isolate Communication Spec](communication.md)** — Check service bindings and standard Bearer Auth middleware.

[SECTION: HOOX GATEWAY ISOLATE PROFILE]

Ø=Ý H O O X G A T E W A Y I S O L A T E P R O F I L E

The **`hoox`** gateway is the public-facing entry point of the trading ecosystem. Running as an ultra-low-latency Cloudflare Worker, the gateway is responsible for authorizing incoming trade signals (TradingView alerts, email routing, manual commands), executing rate-limiting checks, locking transaction trace IDs via Durable Objects to prevent duplicate fills, and routing validated events privately to background compute nodes.

Ø<ß×þ A R C H I T E C T U R A L T O P O L O G Y



&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The gateway's `wrangler.jsonc` defines its private service binding links and resource bounds:

```
  {
    "name": "hoox",
    "main": "src/index.ts",
    "compatibility_date": "2026-05-19",
    "compatibility_flags": ["nodejs_compat"],
    "account_id": "debc6545e63bea36be059cbc82d80ec8",
    "placement": {
      "mode": "smart",
    },
    "vars": {
      "ENVIRONMENT": "production",
    },
```

```
    "kv_namespaces": [
      {
        "binding": "CONFIG_KV",
        "id": "c5917667a21745e390ff969f32b1847d",
      },
      {
        "binding": "SESSIONS_KV",
        "id": "ff70a58b492e45d79880a7a8213c745c",
      },
    ],
    "services": [
      { "binding": "TRADE_SERVICE", "service": "trade-worker" },
      { "binding": "TELEGRAM_SERVICE", "service": "telegram-worker" },
      { "binding": "ANALYTICS_SERVICE", "service": "analytics-worker" },
    ],
    "queues": {
      "producers": [{ "queue": "trade-execution", "binding": "TRADE_QUEUE" }],
    },
    "durable_objects": {
      "bindings": [
        { "name": "IDEMPOTENCY_STORE", "class_name": "IdempotencyStore" },
      ],
    },
    "migrations": [
      {
        "tag": "v1",
        "new_sqlite_classes": ["IdempotencyStore"],
      },
    ],
  }
```

Ø=Ý 2 . E N V I R O N M E N T A L V A R I A B L E S & E N C R Y P

For security, build-time credentials are never stored in plain text. They are bound at deploy time as encrypted secrets:

- **`WEBHOOK_API_KEY`**: The custom authentication passkey expected inside incoming signal payloads.
- **`INTERNAL_KEY_BINDING`**: The shared bearer authorization token used by `requireInternalAuth` middleware to invoke internal V8 isolates.

LOCAL DEVELOPMENT MOCKING (`.DEV.VARS`)

When running tests or starting local Wrangler dev, create a gitignored `.dev.vars` file in the gateway directory:

```
WEBHOOK_API_KEY=dev_webhook_auth_passkey
```

INTERNAL_KEY_BINDING=dev_shared_internal_security_key

Ø=ÞÜ 3 . A P I R O U T E S P E C I F I C A T I O N S

The gateway exposes three public entryways:

A. INGEST SIGNAL WEBHOOK

- **Endpoint**: `/webhook`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "apiKey": "dev_webhook_auth_passkey",
    "exchange": "bybit",
    "action": "LONG",
    "symbol": "BTCUSDT",
    "quantity": 0.01,
    "leverage": 10,
    "idempotencyKey": "uuid-9b1deb4d-3b7d"
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "exchange": "bybit",
    "symbol": "BTCUSDT",
    "action": "LONG",
    "result": { "orderId": "18049284739", "status": "Filled" }
  }
  ```

B. TELEGRAM BOT UPDATE WEBHOOK

- **Endpoint**: `/telegram-webhook`
- **Method**: `POST`
- **JSON Payload**: Standard encrypted update structure forwarded by Telegram's webhook servers.

C. GATEWAY HEALTH DIAGNOSTICS

- **Endpoint**: `/health`
- **Method**: `GET`
- **Success Response (200 OK)**:
  ```json
  {
    "status": "ok",
    "timestamp": 1779261050000,
    "bindings": {
      "d1": "connected",
      "kv": "connected",
      "queue": "active"
    }
  }
  ```

> **Tip:** If exchange APIs experience high latency or go offline, the gateway intercepts the timeout error, serializes the payload, and pushes it to the `TRADE_QUEUE` producer in less than **2 milliseconds**, returning a `"status": "Enqueued"` (202 Accepted) response to TradingView.

Ø=Ý N E X T S T E P S

- **[trade-worker Spec](trade-worker.md)** — Review how trade executions, order math, and margin settings compile on the edge.
- **[D1 Database Operations](../guides/database-ops.md)** — Manage schema migrations, query ledgers, and execute SQL scripts.

[SECTION: TRADE-WORKER ISOLATE PROFILE]

Ø=ÜÈ T R A D E - W O R K E R I S O L A T E P R O F I L E

The **`trade-worker`** is the execution engine of the Hoox trading platform. Deployed as a private compute isolate behind the edge firewall, this worker is responsible for calculating order parameters, executing leverage scaling, performing cryptographic HMAC-SHA256 signature calculations, placing trades on Bybit, Binance, and MEXC, and offloading transactional metrics.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `trade-worker` does not expose a public URL, communicating internally via V8 Service Bindings. Its `wrangler.jsonc` maps out its critical storage, queue, and database hooks:


```
{
  "name": "trade-worker",
  "main": "src/index.ts",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
  "placement": {
    "mode": "smart",
  },
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "trade-data-db",
      "database_id": "c5917667a21745e390ff969f32b1847a",
    },
  ],
  "r2_buckets": [
    { "binding": "REPORTS_BUCKET", "bucket_name": "trade-reports" },
    { "binding": "SYSTEM_LOGS_BUCKET", "bucket_name": "hoox-system-logs" },
  ],
  "kv_namespaces": [
    {
      "binding": "CONFIG_KV",
      "id": "c5917667a21745e390ff969f32b1847d",
    },
  ],
  "queues": {
    "consumers": [
      {
        "queue": "trade-execution",
        "max_batch_size": 10,
        "max_batch_timeout": 1,
```

```
      },
    ],
  },
  "secrets": [
    "INTERNAL_KEY_BINDING",
    "BYBIT_API_KEY",
    "BYBIT_API_SECRET",
    "BINANCE_API_KEY",
    "BINANCE_API_SECRET",
    "MEXC_API_KEY",
    "MEXC_SECRET_BINDING",
  ],
}
```

Ø=Ý 2. THE PROVIDER-BASED `EXCHANGEROUT

To support multiple centralized exchanges with different API schemas while maintaining a clean code structure, `trade-worker` implements a **Provider Composition Pattern**:

A. GENERIC EXCHANGE PROVIDER INTERFACE

The client abstraction is defined inside the shared monorepo package `@jango-blockchained/hoox-shared/types`:

```
export interface IExchangeProvider<TClient, TEnv> {
  readonly name: string;
  createClient(env: TEnv): TClient;
  hasCredentials(env: TEnv): boolean;
}
```

B. DYNAMIC RUNTIME ROUTING

The `ExchangeRouter` evaluates the incoming symbol and parses settings in `CONFIG_KV` in sub-milliseconds:

1. **Default Path**: Routes trades to the default CEX declared in `exchanges:default_routing` (typically `bybit`).
2. **Dynamic Symbol Redirects**: Parses overrides in KV (e.g. `exchanges:routing:SOLUSDT = binance`). If present, the router bypasses Bybit and instantiates the `BinanceProvider` instantly without redeploying code.

Ø=Ý 3. INTERNAL REST API SPECIFICATION

A. PROCESS ORDER PIPELINE

Invoked by the `hoox` gateway or the `TRADE_QUEUE` consumer batch runner.

- **Endpoint**: `/process`
- **Method**: `POST`
- **Headers**: `X-Internal-Auth-Key: <INTERNAL_KEY_BINDING>`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "payload": {
      "exchange": "bybit",
      "action": "LONG",
      "symbol": "BTCUSDT",
      "quantity": 0.005,
      "leverage": 10
    }
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "result": {
      "orderId": "18049284739",
      "status": "Filled",
      "price": 68425.5
    },
    "error": null
  }
  ```

Ø=Þáþ 4. STANDARDIZED EXCEPTION HANDLING

All execution rejects and validation failures are intercepted by the `trade-worker` error middleware and formatted using the shared `Errors` factory from `@jango-blockchained/hoox-shared/errors`:

```
import { Errors } from "@jango-blockchained/hoox-shared/errors";

// 1. Parameter Validation Failure
if (quantity <= 0) {
  return Errors.badRequest("Quantity parameter must be greater than zero.");
}
```

```
  // 2. Exchange Signature Timeout
  if (timestampExpired) {
    return Errors.unauthorized(
      "Timestamp verification failed. Check system NTP sync."
    );
  }

  // 3. API Execution Rejects
  try {
    await exchange.placeOrder(order);
  } catch (err: any) {
    return Errors.internal(`Exchange API Reject: ${err.message}`);
  }
```

> **Tip:** If the exchange API rejects an order due to account rate-limiting, the queue consumer automatically returns a retry flag. Cloudflare Queues will back off and re-route the batch at intervals starting at 30 seconds, protecting your strategy from missed fills.

Ø=Ý N E X T S T E P S

- **[hoox Gateway Profile](hoox.md)** — Review WAF rules and Durable Object idempotency locks.
- **[D1 Database Operations](../guides/database-ops.md)** — Manage Drizzle schemas, migrations, and query operations.

[SECTION: AGENT-WORKER ISOLATE PROFILE]

AGENT WORKER - HOOX AUTONOMOUS AI & RISK MANAGER

Last Updated: May 2026 (Post-Enhancement)

The `agent-worker` serves as the proactive intelligence layer of the Hoox trading ecosystem. Rather than waiting for webhooks, it runs continuously on a cron schedule to monitor portfolio health, enforce risk limits, and optimize position exits.

CORE CAPABILITIES

Feature	Description
#ñþ **Cron-Driven Observation**	Automatically fetch live market data from Binance, Bybit, and MEXC.
Ø=Þáþ **Global Kill Switch**	Calculates the `hoox` gateway from new entries if the `max_daily_drawdown_percent` is breached.
Ø<ß¯ **Dynamic Trailing Stops**	Stores watermarks automatically triggers `CLOSE` payloads if the market reverses.
Ø=Ü¸ **Scale-Out Take Profits**	Detects when target and automatically sends partial close commands to secure gains.
Ø>Ý **AI System Summarization**	Periodically `d1-worker`, analyzes them via LLAMA 3 8B, and sends natural language health reports to Telegram.
Ø<ß **Multi-Provider AI**	Seamlessly sw Anthropic, Google AI, and Azure OpenAI with automatic fallbacks.
Ø>Ýà **Advanced Models**	Supports visi thinking), and code generation models.

ARCHITECTURE & FLOW

1. **Trigger:** Cloudflare® Cron triggers the worker.
2. **State Sync:** Fetches active `OPEN` positions via the `d1-worker`.
3. **Market Pulse:** Pings public exchange APIs for the latest `markPrice`.
4. **Risk Evaluation:** Cross-references current price with KV-stored watermarks and global drawdown limits.
5. **AI Processing:** Uses configured AI provider with automatic fallback chain.

6. **Execution:** Dispatches actions to `trade-worker` (closing positions) and `telegram-worker` (alerts) via internal Service Bindings.

ENDPOINTS & INTERACTIONS

MANAGEMENT ENDPOINTS

`GET /AGENT/CONFIG`

Returns current agent configuration including provider settings.

```
{
  "success": true,
  "config": {
    "defaultProvider": "workers-ai",
    "fallbackChain": ["workers-ai", "openai"],
    "modelMap": { ... },
    "trailingStopPercent": 0.05,
    "takeProfitPercent": 0.10
  }
}
```

`POST /AGENT/CONFIG`

Update agent configuration at runtime.

```
{
  "defaultProvider": "openai",
  "fallbackChain": ["openai", "workers-ai", "anthropic", "google", "azure"],
  "modelMap": {
    "workers-ai": "@cf/meta/llama-3.1-8b-instruct",
    "openai": "gpt-4o-mini-2024-07-18",
    "anthropic": "claude-3-haiku-20240307",
    "google": "gemini-1.5-flash-002",
    "azure": "gpt-4o-mini"
  },
  "timeoutMs": 30000,
  "retryCount": 3
}
```

`GET /AGENT/MODELS`

Returns all available models from Cloudflare Workers AI and external providers.

`POST /AGENT/TEST-MODEL`

Test a specific AI model.

```
{
  "prompt": "Say hello",
  "model": "@cf/meta/llama-3.1-8b-instruct",
  "provider": "workers-ai"
}
```

`GET /AGENT/HEALTH`

Returns health status of all configured AI providers.

```
{
  "success": true,
  "providers": {
    "workers-ai": { "healthy": true, "latency": 150 },
    "openai": { "healthy": true, "latency": 200 }
  }
}
```

AI INTERACTION ENDPOINTS

`POST /AGENT/CHAT`

Send a chat request with automatic provider fallback and **SSE streaming support**.

Request:

```
{
  "messages": [{ "role": "user", "content": "Analyze BTC market sentiment" }],
  "systemPrompt": "You are a professional crypto trading analyst.",
  "temperature": 0.7,
  "maxTokens": 500,
  "stream": true
}
```

Streaming Response (SSE):

```
data: {"content": "Based on current market conditions..."}
data: {"content": " technical indicators suggest..."}
data: [DONE]
```

`POST /AGENT/VISION` (NEW!)

Analyze images with AI vision models. Supports both URL and base64 input.

```
{
  "imageUrl": "https://example.com/chart.png",
  "prompt": "Analyze this price chart and identify key support/resistance levels",
```

```
    "model": "@cf/meta/llama-3.2-11b-vision-instruct"
  }
```

Or with base64:

```
  {
    "imageBase64": "iVBORw0KGgoAAAANSUhEUgAA...",
    "prompt": "What pattern do you see in this chart?"
  }
```

`POST /AGENT/REASONING` (NEW!)

Extended thinking queries with reasoning models like OpenAI o1.

```
  {
    "prompt": "Design a risk management strategy for a $100k portfolio",
    "model": "o1-preview",
    "reasoningEffort": "medium"
  }
```

Response:

```
  {
    "reasoning": "Let me think through this step by step...",
    "answer": "Here's a comprehensive risk management strategy...",
    "model": "o1-preview"
  }
```

`GET /AGENT/USAGE` (NEW!)

Get AI API usage statistics across all providers.

```
  {
    "success": true,
    "usage": {
      "workers-ai": { "requests": 150, "tokens": 45000 },
      "openai": { "requests": 75, "tokens": 22000 }
    }
  }
```

`GET /AGENT/PROMPTS` (NEW!)

List available prompt templates.

```
  {
    "success": true,
    "prompts": ["trading-analyst", "risk-assessor", "market-scanner"]
  }
```

`POST /AGENT/EMBEDDING`

Generate text embeddings using Workers AI embedding models.

```
{
  "text": "Bitcoin price analysis for position sizing",
  "provider": "workers-ai"
}
```

LEGACY ENDPOINTS

`POST /AGENT/RISK-OVERRIDE`

Manually enforce or release risk locks.

```
{
  "action": "engage_kill_switch",
  "reason": "Manual override from dashboard"
}
```

`GET /AGENT/STATUS`

Retrieve the real-time health of the agent and active trailing stops.

CONFIGURATION

KV KEYS

All configuration is stored in `CONFIG_KV` for real-time adjustments.

KV Key	Default	Description
`agent:config`	JSON object	Main provider configuration
`agent:openai_key`	-	OpenAI API key
`agent:anthropic_key`	-	Anthropic API key
`agent:google_key`	-	Google AI API key
`agent:azure_api_key`	-	Azure OpenAI API key
`agent:azure_endpoint`	-	Azure OpenAI endpoint URL

`trade:max_daily_drawdown_percent`	`-5`	Account PnL % that triggers Kill Switch
`trade:kill_switch`	`false`	When `true`, halts all new trades
`trade:watermark:{exchange}:{symbol}:{side}`	N/A	High/low watermark

DEFAULT AGENT CONFIG

```
{
  "defaultProvider": "workers-ai",
  "fallbackChain": ["workers-ai", "openai", "anthropic", "google", "azure"],
  "modelMap": {
    "workers-ai": "@cf/meta/llama-3.1-8b-instruct",
    "openai": "gpt-4o-mini-2024-07-18",
    "anthropic": "claude-3-haiku-20240307",
    "google": "gemini-1.5-flash-002",
    "azure": "gpt-4o-mini"
  },
  "timeoutMs": 30000,
  "retryCount": 3,
  "maxDailyDrawdownPercent": -5,
  "trailingStopPercent": 0.05,
  "takeProfitPercent": 0.1
}
```

SUPPORTED MODELS

WORKERS AI MODELS

Task	Workers AI Model
Chat	`@cf/meta/llama-3.1-8b-instruct`
Vision	`@cf/meta/llama-3.2-11b-vision-instruct`
Reasoning	`@cf/deepseek-ai/deepseek-r1-distill-qwen-32b`
Code	`@cf/qwen/qwen2.5-coder-32b-instruct`
Embeddings	`@cf/baai/bge-base-en-v1.5`
Summarization	`@cf/facebook/bart-large-cnn`

EXTERNAL PROVIDERS

Provider	Models
OpenAI	GPT-4o, GPT-4o-mini, GPT-4 Turbo, o1
Anthropic	Claude 3 Haiku, Sonnet, Opus
Google	Gemini 1.5 Flash, Gemini 1.5 Pro
Azure	GPT-4o, GPT-4o-mini (custom deployment)

AI GATEWAY FEATURES

The AI Gateway provides:

- **Fallback Chain**: Automatically tries providers in order on failure
- **Health Checks**: Providers self-report health status every 5 minutes
- **Retry Logic**: Exponential backoff with configurable max retries
- **Timeout Protection**: Configurable per-request timeout (default 30s)
- **Usage Tracking**: Automatic token and request counting per provider

INTERNAL SERVICE BINDINGS

The `agent-worker` requires the following bindings to operate:

- `D1_SERVICE`: To fetch open positions and system logs.
- `TRADE_SERVICE`: To execute trailing stops and profit-taking.
- `TELEGRAM_SERVICE`: To broadcast AI summaries and emergency alerts.
- `CONFIG_KV`: For dynamic configuration and state.
- `AI`: Workers AI binding for inference.

TESTING

The agent-worker includes comprehensive tests (**171 tests across 33 files**):

```
# Run all tests
bun test

# Run specific test file
bun test tests/ai/providers/openai.test.ts

# Run with coverage
bun test --coverage
```

Test Coverage: >80% across all modules

TEST STRUCTURE

```
tests/
  %%%  ai/
  %      %%%  providers/            # Tests for each AI p
  %      %%%  gateway.test.ts       # AI Gateway fallback
  %      %%%  streaming.test.ts     # SSE streaming test
  %      %%%  prompts.test.ts       # Prompt template tes
  %%%  handlers/                  # Tests for all endpoi
  %%%  middleware/                # Tests for auth, val
  %%%  market/                    # Tests for exchange c
```

```
    %%%   risk/                          #  Tests for risk manag
    %%%   routine/                       #  Tests for housekeepi
    %%%   integration/                   #  End-to-end integrati
```

[SECTION: TELEGRAM-WORKER ISOLATE PROFILE]

Ø=Ü¬ T E L E G R A M - W O R K E R I S O L A T E P R O F I L

The **`telegram-worker`** serves as the dynamic communicator of the Hoox trading ecosystem. Running as an isolated edge microservice, it parses incoming chat commands forwarded from Telegram's webhooks, executes retrieval-augmented generation (RAG) queries via Vectorize, analyzes screenshots using AI vision models, and dispatches real-time HTML/Markdown formatting order alerts to your mobile.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `telegram-worker` mounts multiple storage and vector search services to implement AI-powered chatbot features. Its `wrangler.jsonc` specifies:


```
{
  "name": "telegram-worker",
  "main": "src/index.ts",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
  "placement": {
    "mode": "smart",
  },
  "kv_namespaces": [
    {
      "binding": "CONFIG_KV",
      "id": "c5917667a21745e390ff969f32b1847d",
    },
  ],
  "r2_buckets": [
    {
      "binding": "UPLOADS_BUCKET",
      "bucket_name": "user-uploads",
    },
  ],
  "vectorize": [
    {
      "binding": "VECTORIZE_INDEX",
      "index_name": "rag-index",
    },
  ],
  "ai": {
    "binding": "AI",
  },
  "secrets": [
    "INTERNAL_KEY_BINDING",
```

```
      "TG_BOT_TOKEN_BINDING",
      "TELEGRAM_CHAT_ID_DEFAULT",
      "TELEGRAM_WEBHOOK_SECRET",
    ],
  }
```

Ø=Ý 2. ENVIRONMENTAL VARIABLES & ENCRYP

- **`TG_BOT_TOKEN_BINDING`**: Private HTTP bot token generated by `@BotFather`.
- **`TELEGRAM_CHAT_ID_DEFAULT`**: Your authorized Chat ID acting as a fallback receiver and admin lock.
- **`TELEGRAM_WEBHOOK_SECRET`**: A secure, random token appended to the webhook URL path to validate that the request originated from Telegram’s servers.
- **`INTERNAL_KEY_BINDING`**: Shared key used to validate calls from `hoox` or `trade-worker`.

LOCAL DEVELOPMENT MOCKING (`.DEV.VARS`)

```
TG_BOT_TOKEN_BINDING=mock_telegram_bot_token
TELEGRAM_CHAT_ID_DEFAULT=987654321
TELEGRAM_WEBHOOK_SECRET=local_secure_route_token
INTERNAL_KEY_BINDING=dev_shared_internal_security_key
```

Ø=Ý 3. INTERNAL REST API SPECIFICATION

A. DISPATCH NOTIFICATION ENDPOINT

- **Endpoint**: `/alert`
- **Method**: `POST`
- **Headers**: `X-Internal-Auth-Key: <INTERNAL_KEY_BINDING>`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "payload": {
      "chatId": "987654321",
            "message": "<b>Ø=ÜÈ Bybit LONG Filled!</b>\n<code>$68,425.50</code>\nQuantity: <code>0.005</code>",
      "parseMode": "HTML"
    }
  }
  ```
- **Success Response (200 OK)**:

```json
{
  "success": true,
  "result": { "messageId": 482904 },
  "error": null
}
```

B. TELEGRAM INGRESS WEBHOOK ROUTE

Receives chat commands, `/start` signals, and image binaries.

- **Endpoint**: `/telegram/<TELEGRAM_WEBHOOK_SECRET>`
- **Method**: `POST`
- **JSON Payload**: Standard Telegram update JSON schema.
- **Access Control**: The worker extracts the incoming `message.chat.id`. If it does not match your authorized `TELEGRAM_CHAT_ID_DEFAULT` secret, the payload is silently dropped with a `200 OK` return to Telegram to prevent malicious commands.

Ø>Ýà 4. AI-POWERED CHATBOT & RAG MECHANI

The bot does not just return static responses. When you send a natural language question (e.g., _"What is my average entry price on SOL?"_):

1. **Embedding Generation**: The worker calls `env.AI` to generate high-dimensional text embeddings for your prompt using the `@cf/baai/bge-base-en-v1.5` model.
2. **Vector Query**: Passes the vector to `env.VECTORIZE_INDEX` to retrieve semantically related transaction rows and market summaries from past trades.
3. **Context Construction**: Formats the matched history into a rich LLM context window.
4. **LLM Inference**: Invokes LLaMA-3 instruct via `env.AI` using your multi-provider API keys to generate a highly informed financial summary.

> **Tip:** Secure your bot webhook instantly after deployment: `hoox deploy telegram-webhook`. The CLI automatically queries your secrets, makes the HTTP setup call to Telegram's servers, and validates the TLS tunnel!

Ø=Ý NEXT STEPS

- **[hoox Gateway Profile](hoox.md)** — Review how incoming commands route to the gateway.

- **[Setting Up Telegram Bot Alerts](../../tutorials/telegram-bot.md)** — Step-by-step tutorial for BotFather configuration.

[SECTION: D-WORKER ISOLATE PROFILE]

Ø=ÝÄþ D 1 - W O R K E R I S O L A T E P R O F I L E

The **`d1-worker`** is the data routing hub of the Hoox trading platform. Deployed as an isolated, private micro-worker, it acts as a centralized SQL execution proxy. By encapsulating database interactions behind secure Service Bindings, it allows other lightweight compute workers to execute parameterized queries, trigger transactional batch operations, and retrieve structured dashboard telemetry without direct database driver overhead.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `d1-worker` binds directly to the production SQLite database (`trade-data-db`) and does not expose any public endpoints:


```
{
  "name": "d1-worker",
  "main": "src/index.ts",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
  "placement": {
    "mode": "smart",
  },
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "trade-data-db",
      "database_id": "c5917667a21745e390ff969f32b1847a",
    },
  ],
  "kv_namespaces": [
    {
      "binding": "CONFIG_KV",
      "id": "c5917667a21745e390ff969f32b1847d",
    },
  ],
  "secrets": ["INTERNAL_KEY_BINDING"],
}
```

Ø=Ý 2 . I N T E R N A L R E S T A P I S P E C I F I C A T I O N

Every endpoint is secured via `requireInternalAuth` and expects the `X-Internal-Auth-Key`

header.

A. EXECUTE SINGLE SQL QUERY

- **Endpoint**: `/query`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "query": "SELECT created_at, symbol, action, price FROM trades WHERE symbol = ? ORDER BY created_at DESC LIMIT ?",
    "params": ["BTCUSDT", 5]
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "results": [
      {
        "created_at": 1779261050000,
        "symbol": "BTCUSDT",
        "action": "LONG",
        "price": 68425.5
      }
    ],
    "meta": { "rows_read": 1, "rows_written": 0, "duration": 4.2 }
  }
  ```

B. EXECUTE TRANSACTIONAL BATCH OPERATIONS

Allows running multiple statements atomically in a single network trip, reducing latency.

- **Endpoint**: `/batch`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "queries": [
      {
        "query": "INSERT INTO trades (id, symbol, price) VALUES (?, ?, ?)",
  ```

```
        "params": ["trade-1", "BTCUSDT", 68000]
      },
      {
        "query": "UPDATE positions SET size = size + ? WHERE symbol = ?",
        "params": [0.005, "BTCUSDT"]
      }
    ]
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "results": [{ "meta": { "changes": 1 } }, { "meta": { "changes": 1 } }]
  }
  ```

C. DASHBOARD TELEMETRY STATISTICS

Calculates aggregated win ratios, active positions size, and time-series P&L.

- **Endpoint**: `/api/dashboard/stats`
- **Method**: `GET`
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "stats": {
      "totalTrades": 1048,
      "winRate": 64.2,
      "totalPnlUSDT": 18340.5,
      "activePositionsCount": 2,
      "dailyTradesCount": 14
    }
  }
  ```

Ø=Þáþ 3 . S E C U R I T Y & S Q L I N J E C T I O N P R O T E C

To protect financial transaction ledgers and portfolios against SQL injection attacks, `d1-worker` enforces strict development rules:

- **Parameterized Bindings**: All inputs must utilize parameterized placeholders (`?` or `?1`) mapped to `env.DB.prepare().bind()`. **Never** concatenate raw request strings directly into SQL statements.
- **Access Isolation**: The database does not exist publicly. By binding D1 solely to this worker and accessing it via V8 Service Bindings, you prevent external crawlers or bots from querying database nodes directly.

> **Tip**: If you are extending schemas or adding tables, generate migration scripts locally using Drizzle: `hoox db migrate --remote`. This keeps edge schema histories atomic and securely tracked.

Ø=Ý N E X T S T E P S

- **[System Storage Architecture](../architecture/storage.md)** — Review SQLite properties and R2 bucketing pipelines.
- **[Database Operations Manual](../guides/database-ops.md)** — Learn commands to run query ledgers and restore backups.

[SECTION: WEB-WALLET-WORKER ISOLATE PROFILE]

Ø<ß W E B 3 - W A L L E T - W O R K E R I S O L A T E P R O

The **`web3-wallet-worker`** is the on-chain gateway of the Hoox trading ecosystem. Running as an isolated private micro-worker, this service is responsible for securely managing EVM mnemonics and private keys (bound as encrypted Workers Secrets), querying multi-chain gas limits and token balances, executing native/ERC-20 transfers, and signing smart contract swap payloads (e.g. Uniswap/1inch routers) via JSON-RPC providers.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `web3-wallet-worker` does not expose a public URL, communicating internally via V8 Service Bindings. Its `wrangler.jsonc` specifies:

```
  {
    "name": "web3-wallet-worker",
    "main": "src/index.ts",
    "compatibility_date": "2026-05-19",
    "compatibility_flags": ["nodejs_compat"],
    "account_id": "debc6545e63bea36be059cbc82d80ec8",
    "vars": {
      "DEFAULT_CHAIN": "ethereum",
    },
    "kv_namespaces": [
      {
        "binding": "CONFIG_KV",
        "id": "c5917667a21745e390ff969f32b1847d",
      },
    ],
    "secrets": [
      "INTERNAL_KEY_BINDING",
      "WALLET_MNEMONIC_SECRET",
      "WALLET_PK_SECRET",
      "RPC_PROVIDER_URL",
    ],
  }
```

Ø=Ý 2 . E N V I R O N M E N T A L V A R I A B L E S & E N C R Y P

- **`WALLET_PK_SECRET`**: Encrypted private key used for single-account execution.
- **`WALLET_MNEMONIC_SECRET`**: Encrypted 12 or 24-word HD wallet seed phrase used to derive multiple accounts.
- **`RPC_PROVIDER_URL`**: High-availability HTTP Ethereum / EVM RPC provider (e.g.,

Infura, Alchemy, or QuickNode).
- **`INTERNAL_KEY_BINDING`**: Shared key used to validate calls from internal compute nodes.

LOCAL DEVELOPMENT MOCKING (`.DEV.VARS`)

```
WALLET_PK_SECRET=0x0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
WALLET_MNEMONIC_SECRET="abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about"
RPC_PROVIDER_URL=http://localhost:8545
INTERNAL_KEY_BINDING=dev_shared_internal_security_key
```

Ø=Ý 3. INTERNAL REST API SPECIFICATION

A. EXECUTE ON-CHAIN TRANSACTION

- **Endpoint**: `/process`
- **Method**: `POST`
- **Headers**: `X-Internal-Auth-Key: <INTERNAL_KEY_BINDING>`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "payload": {
      "action": "sendTransaction",
      "chain": "arbitrum",
      "to": "0x6b175474e89094c44da98b954eedeac495271d0f",
      "value": "0.05",
      "data": "0xa9059cbb000000000000000000000000...",
      "gasLimit": 100000
    }
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "result": {
      "txHash": "0x53a9284739ebfd10482da73cbcfd10482da73cbcfd10482da73cbcfd10482ab",
      "nonce": 142,
      "gasUsed": 64205,
      "effectiveGasPrice": "24000000000"
    },
    "error": null
  }
```

```

---

**B. QUERY TOKEN BALANCE**

- **Endpoint**: `/process`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "payload": {
      "action": "getBalance",
      "chain": "polygon",
      "address": "0x6b175474e89094c44da98b954eedeac495271d0f",
      "tokenAddress": "0xc2132d05d31c914a87c6611c10748aeb04b58e8f"
    }
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "result": {
      "balance": "1485.50",
      "symbol": "USDT",
      "decimals": 6
    },
    "error": null
  }
  ```

Ø=Þáþ 4. ON-CHAIN SECURITY BEST PRACTICE

Operating hot wallets on public blockchain networks introduces extreme security vectors:

- **Harden Private Keys**: **Never** write keys to wrangler config files or print them in telemetry logs. Always provision keys via encrypted Cloudflare Secrets.
- **Gas Price Limit Traps**: To prevent severe loss during network congestion or flash crashes, the worker enforces a **gas limit trap**—if current network gas price exceeds your KV configured limit (`web3:max_gas_price_gwei`), transactions are dropped before signing to prevent massive fee consumption.
- **Isolate Access**: All calls must originate internally via Service Bindings. The
  ```

wallet worker does not bind to public ports, meaning external scrapers cannot send raw transaction payloads or try to brute-force auth codes.

> **Tip:** Testing on-chain logic locally? Use the Docker runtime stack (`hoox dev start --runtime docker`) to launch an isolated Hardhat/Anvil node container and test private wallet swaps on a simulated local EVM fork safely!

Ø=Ý NEXT STEPS

- **[trade-worker Profile](trade-worker.md)** — Review how execution orders route transactions to EVM wallet nodes.
- **[D1 Database Operations](../guides/database-ops.md)** — Manage your SQLite schemas and sync positions logs.

[SECTION: EMAIL-WORKER ISOLATE PROFILE]

Ø=Üç E M A I L - W O R K E R I S O L A T E P R O F I L E

The **`email-worker`** is an ancillary signal ingestion plugin. Deployed as a private compute isolate, this service is responsible for intercepting trading signals distributed via email lists, newsletters, or alert emails (e.g. from TradingView or custom notification pipelines). It validates domain security (SPF/DKIM), parses message subjects and bodies using dynamic **Regular Expressions**, and routes the extracted signals privately to `trade-worker` via V8 Service Bindings.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `email-worker` does not expose any public endpoints, connecting internally to active executors and metrics collectors:

```
{
  "name": "email-worker",
  "main": "src/index.ts",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
  "vars": {
    "USE_IMAP": "true",
  },
  "kv_namespaces": [
    {
      "binding": "CONFIG_KV",
      "id": "c5917667a21745e390ff969f32b1847d",
    },
  ],
  "services": [
    { "binding": "TRADE_SERVICE", "service": "trade-worker" },
    { "binding": "ANALYTICS_SERVICE", "service": "analytics-worker" },
  ],
  "secrets": [
    "INTERNAL_KEY_BINDING",
    "EMAIL_HOST_BINDING",
    "EMAIL_USER_BINDING",
    "EMAIL_PASS_BINDING",
    "MAILGUN_API_KEY",
  ],
}
```

Ø=Ý 2 . E N V I R O N M E N T A L V A R I A B L E S & E N C R Y P

- **`EMAIL_HOST_BINDING`**: POP3/IMAP mailbox server address (e.g., `imap.gmail.com`).
- **`EMAIL_USER_BINDING`**: Dedicated signals email mailbox username.
- **`EMAIL_PASS_BINDING`**: Secure app-specific password.
- **`MAILGUN_API_KEY`**: Key used to validate incoming SMTP webhooks if using Mailgun routing.
- **`INTERNAL_KEY_BINDING`**: Shared key used to validate calls from internal compute nodes.

Ø=ÝÃþ 3. REGEX PARSING MECHANICS IN V8

When a new email is scanned or received via webhook, the worker evaluates the content against regular expression arrays stored in `CONFIG_KV`:

Configuration Key	Type	Default Pattern	Parser Purpose
`email:scan_subject`	`string`	`^TRADE SIGNAL:.*`	Resolves whether the email should be parsed or ignored.
`email:coin_pattern`	`string`	`(BTC|ETH|SOL|LINK)`	Matches uppercase asset tokens in the email body.
`email:action_pattern`	`string`	`(BUY|SELL|LONG|SHORT)`	Resolves buy/sell execution parameters.
`email:quantity_multiplier`	`number`	`1.0`	Coefficient applied to parsed quantities to scale positions.

```
  // Under the hood regex parsing logic
  const subjectRegex = new RegExp(env.CONFIG_KV.get("email:scan_subject"));
  const coinRegex = new RegExp(env.CONFIG_KV.get("email:coin_pattern"));
  const actionRegex = new RegExp(env.CONFIG_KV.get("email:action_pattern"));

  if (subjectRegex.test(email.subject)) {
    const asset = email.body.match(coinRegex)?.[0]; // Extracts "BTC"
    const action = email.body.match(actionRegex)?.[0]; // Extracts "LONG"
    // Routes to trade-worker...
  }
```

Ø=Ý 4. API INTERFACE SPECIFICATION

While the worker primarily runs background Cron scan loops, it exposes two ingestion endpoints for webhook integrations (e.g. Mailgun/SendGrid):

A. MAILGUN WEBHOOK ROUTER

- **Endpoint**: `/webhook/mailgun`
- **Method**: `POST`
- **Headers**: `Mailgun-Signature`, `Mailgun-Timestamp`, `Mailgun-Token`
- **Security**: The worker computes the HMAC-SHA256 signature of the timestamp and token using your `MAILGUN_API_KEY` secret. If the calculated signature doesn't match the header, the request is instantly rejected (401 Unauthorized).

B. DIRECT JSON PAYLOAD INGESTION

Used primarily by administrative automation tools or local testing scripts.

- **Endpoint**: `/process`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "payload": {
      "sender": "alerts@tradingview.com",
      "subject": "TRADE SIGNAL: Breakout SOL",
      "body": "Action: LONG, Asset: SOLUSDT, Size: 1.5"
    }
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "result": { "symbol": "SOLUSDT", "action": "LONG", "quantity": 1.5 },
    "error": null
  }
  ```

Ø=Þáþ 5 . D O M A I N I N G R E S S P R O T E C T I O N S : S P F

To protect your wallet against email spoofing attacks:

1. **DKIM Check**: Validates the cryptographic DKIM signature in the email header to prove the message body was not altered in transit.
2. **SPF Check**: Matches the sending server's IP address against the DNS TXT SPF record

of the authorized domain, dropping forged emails before parsing.

Ø=Ý N E X T S T E P S

- **[trade-worker Profile](trade-worker.md)** — Review how execution orders route transactions to EVM wallet nodes.
- **[System Storage Architecture](../architecture/storage.md)** — Review SQLite properties and R2 bucketing pipelines.

[SECTION: ANALYTICS-WORKER ISOLATE PROFILE]

Ø=ÜÊ A N A L Y T I C S - W O R K E R I S O L A T E P R O F I

The **`analytics-worker`** is the observability engine of the Hoox trading platform. Deployed as a private internal microservice, it aggregates time-series metrics, database query latencies, execution performance ratios, and API status codes across all V8 isolates. By translating incoming events and writing them to **Cloudflare Analytics Engine**, it provides the backend telemetry used to draw live charts in the Next.js Dashboard.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `analytics-worker` binds directly to Cloudflare’s **Analytics Engine** dataset and does not expose any public endpoints:

```
{
  "name": "analytics-worker",
  "main": "src/index.ts",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
  "analytics_engine_datasets": [
    {
      "binding": "ANALYTICS_ENGINE",
      "dataset": "hoox_telemetry",
    },
  ],
  "kv_namespaces": [
    {
      "binding": "CONFIG_KV",
      "id": "c5917667a21745e390ff969f32b1847d",
    },
  ],
  "secrets": [
    "INTERNAL_KEY_BINDING",
    "CLOUDFLARE_API_TOKEN", // Required to run SQL queries against Analytics datasets
  ],
}
```

Ø=Ý 2 . E N V I R O N M E N T A L V A R I A B L E S & E N C R Y P

- **`CLOUDFLARE_API_TOKEN`**: A secure token with `Account.Analytics` read permissions, allowing the worker to query stored datasets.

- **`INTERNAL_KEY_BINDING`**: Shared key used to validate calls from other V8 isolates.

Ø=Ý 3. INTERNAL REST API SPECIFICATION

Every endpoint is secured via `requireInternalAuth` and expects the `X-Internal-Auth-Key` header.

A. TRACK TELEMETRY EVENT

Invoked by other workers (like `hoox` or `trade-worker`) immediately upon completing an action.

- **Endpoint**: `/track/api-call`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "payload": {
      "worker": "trade-worker",
      "endpoint": "/webhook",
      "latencyMs": 24.5,
      "success": true
    }
  }
  ```

UNDER-THE-HOOD ANALYTICS ENGINE WRITE

When received, the worker formats and writes a time-series data point using standard **Blobs** (metadata strings) and **Doubles** (numeric values):

```
  env.ANALYTICS_ENGINE.writeDataPoint({
    blobs: [
      payload.worker, // Blob 1: Worker Name
      payload.endpoint, // Blob 2: Endpoint Path
      payload.success ? "1" : "0", // Blob 3: Success state
    ],
    doubles: [
      payload.latencyMs, // Double 1: Latency in milliseconds
    ],
    indexes: [
      payload.requestId, // Custom index for distributed tracing
    ],
  });
```

- **Response (200 OK)**:
  ```json
  { "success": true }
  ```

B. QUERY METRICS (SQL INTERFACE)

Used primarily by the Next.js Dashboard to extract data points for chart rendering.

- **Endpoint**: `/query`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "query": "SELECT sum(double1) as total_latency, count() as total_calls FROM hoox_telemetry WHERE blob1 = ? AND timestamp >= now() - INTERVAL '1' DAY",
    "params": ["trade-worker"]
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "results": [{ "total_latency": 10482.5, "total_calls": 428 }]
  }
  ```

Ø=ÜÈ 4. DASHBOARD VISUAL INTEGRATIONS

The metrics written to `hoox_telemetry` are queried by the dashboard to render:

1. **System Health Indicators**: Real-time error spikes trigger warning banners in under 2 seconds.
2. **Isolate Latency Charts**: Visual line graphs showing V8 processing speeds vs exchange API transit speeds.
3. **Volume Load Heatmaps**: Visualizes peak trading hours and signal traffic volumes globally.

> **Tip:** Testing analytics locally? Local Wrangler dev automatically intercepts

`writeDataPoint` calls and logs the parsed Blobs and Doubles straight to your terminal standard output, ensuring easy debugging!

Ø=Ý N E X T S T E P S

- **[System Observability Guides](../deployment/monitoring.md)** — Deepen your understanding of time-series logging and metrics.
- **[trade-worker Profile](trade-worker.md)** — Review how execution latency details are generated and offloaded.

[SECTION: REPORT-WORKER ISOLATE PROFILE]

Ø=ÜÄ R E P O R T - W O R K E R I S O L A T E P R O F I L E

The **`report-worker`** is the automated document compiler of the Hoox trading platform. Deployed as a scheduled cron isolate, this worker runs twice daily (at 06:00 and 18:00 UTC) to pull D1 transactional stats, construct a highly styled HTML5 portfolio report, spin up a serverless **Cloudflare Browser Rendering** Chrome instance to compile the page into a PDF buffer, offload the file to R2, and push a private download link to your Telegram.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The `report-worker` mounts storage buckets, notification bindings, and is triggered by Cloudflare's background cron engine:

```
{
  "name": "report-worker",
  "main": "src/index.ts",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
  "placement": {
    "mode": "smart",
  },
  "triggers": {
    "crons": ["0 6 * * *", "0 18 * * *"], // Runs twice daily
  },
  "r2_buckets": [
    {
      "binding": "REPORTS_BUCKET",
      "bucket_name": "trade-reports",
    },
  ],
  "services": [
    { "binding": "D1_SERVICE", "service": "d1-worker" },
    { "binding": "TELEGRAM_SERVICE", "service": "telegram-worker" },
  ],
  "secrets": [
    "INTERNAL_KEY_BINDING",
    "CF_API_TOKEN", // Required to call Browser Rendering REST API
  ],
}
```

Ø=Ý 2 . E N V I R O N M E N T A L V A R I A B L E S & E N C R Y P

- **`CF_API_TOKEN`**: Your Cloudflare API Token with `Account.Browser Rendering` and `Account.R2` write permissions.
- **`INTERNAL_KEY_BINDING`**: Shared key used to validate calls from other V8 isolates.

Ø<ß 3. BROWSER RENDERING & PDF PRINT PI

When the Cron schedule triggers, `report-worker` runs the following programmatic pipeline:

STEP 1: DATA AGGREGATION & HTML COMPILATION

The worker pings `D1_SERVICE` to retrieve P&L, win rate, and total daily fees, inserting the metrics into a styled HTML5 template utilizing Tailwind CSS and CSS Grid styling:

```
  <div class="card">
    <h2>Daily P&L</h2>
    <p class="pnl-green">+$1,234.50</p>
  </div>
```

STEP 2: CLOUDFLARE CHROME ISOLATE PRINT

Pushes the HTML template to Cloudflare's headless Chrome rendering pool via the REST API:

```
  const response = await fetch(
`https://api.cloudflare.com/client/v4/accounts/${env.ACCOUNT_ID}/browser-rendering/pdf`,
    {
      method: "POST",
      headers: {
        Authorization: `Bearer ${env.CF_API_TOKEN}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        html: htmlContent,
        options: {
          format: "A4",
          printBackground: true,
          margin: { top: "1cm", bottom: "1cm", left: "1cm", right: "1cm" },
        },
      }),
    }
  );
```

STEP 3: R2 STORAGE & EXPIRATION

1. The API compiles the page and returns a raw PDF binary stream.
2. The worker uploads the stream to the `REPORTS_BUCKET` R2 storage bucket using a unique, date-hashed key:
   `reports/daily-pnl-2026-05-19.pdf`
3. A lifecycle rule on the R2 bucket automatically purges reports older than 30 days to maintain storage cleanliness.

STEP 4: DISPATCH TELEGRAM ALERT

Calls `TELEGRAM_SERVICE` to send the link:

```
  await env.TELEGRAM_SERVICE.fetch("https://telegram-worker/alert", {
    method: "POST",
    headers: { "X-Internal-Auth-Key": env.INTERNAL_KEY_BINDING },
    body: JSON.stringify({
      chatId: env.TELEGRAM_CHAT_ID_DEFAULT,
      message:
                " < b >Ø=ÜÄ  D a i l y  P & L  P D F  R e p o r t  C o m p i l e d ! < / b
href='https://reports.cryptolinx.workers.dev/daily-pnl-2026-05-19.pdf'>Download PDF</a>",
    }),
  });
```

> **Tip:** If `CF_API_TOKEN` is not configured, the worker gracefully fails by compiling a rich text-only P&L summary and pushing it directly via Telegram instead of generating a PDF, guaranteeing continuous operations without hard crashes.

Ø=Ý N E X T S T E P S

- **[telegram-worker Profile](telegram-worker.md)** — Review push alert configurations and webhook tunnels.
- **[D1 Database Operations](../guides/database-ops.md)** — Manage Drizzle schemas and execute custom queries.

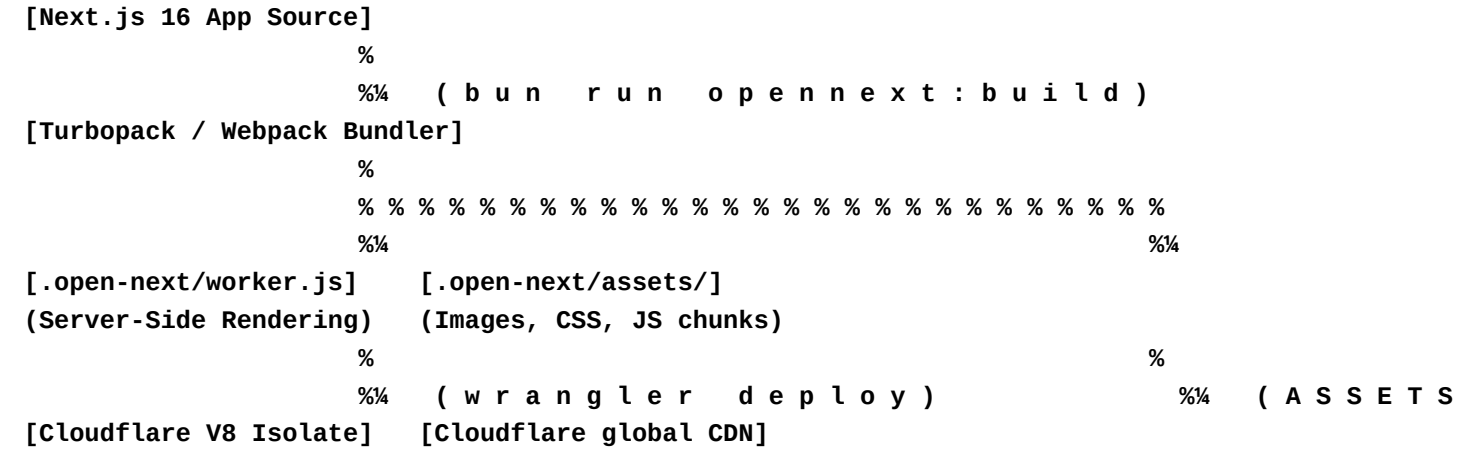
[SECTION: NEXT.JS DASHBOARD & OPENNEXT ISOLATE]

Ø=Ý¥þ N E X T . J S D A S H B O A R D & O P E N N E X T

The **Hoox Dashboard** is the web-based command center of the trading platform. Rather than running on a traditional server, the dashboard is a **Next.js 16 application** compiled using the **OpenNext adapter** and hosted natively on Cloudflare Workers. This edge-first layout allows for real-time portfolio monitoring, dynamic form-based settings editing, and interactive kill-switch controls at microsecond speeds.

Ø<ß×þ N E X T . J S T O C L O U D F L A R E E D G E A R C H I T E

When deploying the dashboard, the monorepo uses `@opennextjs/cloudflare` to convert standard Node.js server-side features into Edge-compliant JavaScript:



- **Server Isolate** (`.open-next/worker.js`): Handles Server-Side Rendering (SSR), React Server Components (RSC) hydration, API routes, and cookies encryption.
- **Static Assets** (`.open-next/assets/`): Houses all static assets, which are bound to the worker via **ASSETS bindings** to serve static pages at sub-millisecond CDN latency.

&¡ 1 . D E C L A R E D W R A N G L E R C O N F I G U R A T I O N S

The dashboard's `wrangler.jsonc` maps out its internal service links to extract D1 databases metrics and agent status updates:

```
{
  "name": "hoox-dashboard",
  "main": ".open-next/worker.js",
  "compatibility_date": "2026-05-19",
  "compatibility_flags": ["nodejs_compat"],
  "account_id": "debc6545e63bea36be059cbc82d80ec8",
```

```
    "assets": {
      "directory": ".open-next/assets",
      "binding": "ASSETS",
    },
    "kv_namespaces": [
      {
        "binding": "CONFIG_KV",
        "id": "c5917667a21745e390ff969f32b1847d",
      },
    ],
    "services": [
      { "binding": "D1_SERVICE", "service": "d1-worker" },
      { "binding": "AGENT_SERVICE", "service": "agent-worker" },
      { "binding": "TELEGRAM_SERVICE", "service": "telegram-worker" },
    ],
    "vars": {
      "D1_SERVICE_URL": "https://d1-worker.cryptolinx.workers.dev",
      "AGENT_SERVICE_URL": "https://agent-worker.cryptolinx.workers.dev",
      "TELEGRAM_SERVICE_URL": "https://telegram-worker.cryptolinx.workers.dev",
    },
    "secrets": [
      "DASHBOARD_USER",
      "DASHBOARD_PASS",
      "SESSION_SECRET",
      "INTERNAL_KEY_BINDING",
    ],
  }
```

Ø=Ý 2 . E N V I R O N M E N T A L V A R I A B L E S & E N C R Y P

- **`DASHBOARD_USER`** & **`DASHBOARD_PASS`**: Administrative credentials required to access the visual panel.
- **`SESSION_SECRET`**: A secure 32-character encryption key used to cryptographically sign session cookies at the edge.
- **`INTERNAL_KEY_BINDING`**: Shared key used to validate calls to the `d1-worker` and `agent-worker` service bindings.

LOCAL DEVELOPMENT MOCKING (`.ENV.LOCAL`)

Create a gitignored `.env.local` file inside `workers/dashboard/` for local Next.js runs:

```
DASHBOARD_USER=admin
DASHBOARD_PASS=admin_dev_passkey_183
SESSION_SECRET=abandon_abandon_abandon_abandon_32
```

Ø<ß›þ 3. SCHEMA-DRIVEN SETTINGS SYSTEM

To allow operators to extend the platform's settings without touching React code, the dashboard implements a **schema-driven configuration manager** parsed dynamically from `config.schema.json`:

```
{
  "sections": [
    {
      "id": "risk",
      "title": "Risk Management",
      "fields": [
        {
          "key": "trade:max_daily_drawdown_percent",
          "label": "Max Daily Drawdown (%)",
          "type": "number",
          "default": 5,
          "category": "risk"
        }
      ]
    }
  ]
}
```

FORM SUBMISSION FLOW

1. When you save the settings form, the Next.js server actions intercept the payload.
2. The server action iterates over the JSON schema keys and writes the values directly to the bound `CONFIG_KV` namespace.
3. The changes propagate globally to your edge executors in under 10 seconds.

Ø=Þ¢ 4. PRODUCTION BUILD & ROLLOUT RUNBO

To compile and deploy the Next.js dashboard to Cloudflare:

```
# 1. Navigate to the dashboard directory
cd workers/dashboard

# 2. Build the application using OpenNext
bun run opennext:build

# 3. Deploy the compiled bundles to Cloudflare Workers
bun run opennext:deploy
```

> **Warning:** Framer Motion components and interactive visual elements used in Next.js pages **must** declare the `'use client'` directive at the top of the file. Under

OpenNext, client-side pages cannot export metadata directly—move metadata definitions to separate `metadata.ts` files!

Ø=Ý N E X T S T E P S

- **[agent-worker Profile](agent-worker.md)** — Review AI chat streaming and risk evaluation structures.
- **[d1-worker Profile](d1-worker.md)** — Check REST schemas that feed database metrics to the dashboard.

[SECTION: PRODUCTION DEPLOYMENT RUNBOOK]

Ø=Þ€ P R O D U C T I O N D E P L O Y M E N T R U N B O O K

Deploying the Hoox trading ecosystem to production requires absolute operational rigor. Because the platform executes real-world trades using private capital, every V8 isolate, environment binding, and routing domain must be locked down, optimized for speed, and insulated against external failures.

This runbook guides you through production prerequisites, manual and automated deployment commands, custom domain routing, and edge hardening strategies.

Ø<ßÁ 1 . P R O D U C T I O N R O L L O U T P R E - F L I G H T C H

Before rolling out updates to Cloudflare’s live edge networks:

- **API Token Scoping**: Confirm your active Cloudflare API Token inherits the strict minimal permission sets (`Account.D1: Edit`, `Account.KV: Edit`, `Account.Queues: Edit`, `Account.Workers: Edit`, and `Zone.DNS: Edit` if mapping custom routing paths).
- **Workspace Diagnostic Sweep**: Run the automated pre-flight check to verify that all git submodules, local dependencies, and TypeScript variables compile without warnings:
  ```bash
  hoox check prerequisites
  hoox test
  ```
- **Exchange Credentials Check**: Ensure you have created dedicated exchange API keys with **Withdrawal permissions turned OFF (disabled)** to enforce least-privilege security.

&¡ 2 . P R O D U C T I O N R O L L O U T I N V O C A T I O N S

The Hoox CLI automates the entire deployment sequence, compiling the shared libraries, deploying database schemas, synchronizing configuration manifests, and uploading workers in the correct dependency sequence:

```
# 1. Execute full sequenced deployment
hoox deploy all --auto

# 2. Deploy a single specific worker (e.g. after a trade execution update)
hoox deploy worker trade-worker
```

DASHBOARD OPENNEXT COMPILATION

```
# Build and deploy the Next.js Dashboard via OpenNext
hoox deploy dashboard
```

This command runs Turbopack, bundles server-side assets into `.open-next/worker.js`, maps static files to the `ASSETS` CDN binding, and uploads the dashboard to Cloudflare.

Ø<ß 3. CUSTOM DOMAIN & ROUTING MAPPING

By default, deployed workers are assigned subdomains under Cloudflare's shared domain (e.g., `https://hoox.alpha-trading.workers.dev`).

For production, it is highly recommended to map your public gateway and dashboard to a **custom domain** under your own Cloudflare zone. This reduces DNS lookup latencies and enables custom SSL and WAF configurations.

To map custom routes, add the `routes` array inside your worker's `wrangler.jsonc` file:

```
// In workers/hoox/wrangler.jsonc
{
  "routes": [
    {
      "pattern": "api.my-trading-empire.com/webhook",
      "custom_domain": true,
    },
  ],
}
```

Deploying this configuration automatically updates your Cloudflare DNS zone records, maps the domain to your V8 isolate, and registers a universal SSL certificate near-instantaneously.

Ø=Þáþ 4. EDGE ISOLATE HARDENING STRATEGI

To protect your live production deployments from attacks and latency spikes:

A. ENABLE SMART PLACEMENT

Verify that every execution worker contains the smart placement var inside its `wrangler.jsonc`. This shifts the CPU isolate geographically close to exchange servers (e.g. Frankfurt for Bybit, Tokyo for Binance):

```
"placement": {
```

```
  "mode": "smart"
}
```

B. RESTRICT CORS ORIGIN POLICIES

The public `/webhook` route inside the `hoox` gateway must **only** accept POST requests from authorized endpoints. Disable all CORS headers to prevent cross-origin script injections from browsers.

C. DEPLOY WAF RATE LIMIT TRAPS

Use the Hoox CLI to provisionZone-level rate limiters on Cloudflare’s global edge network, dropping packet floods before they load the V8 isolates:

```
# Provision a strict rate-limit rule (e.g., max 10 requests/minute to /webhook)
hoox waf configure --limit-requests
```

> **Tip:** Made an emergency change? You don't need a full rebuild. If the change was a configuration setting or a trade symbol routing change, simply update the KV store: `hoox config kv set <key> <value>`. The update will propagate globally in under 10 seconds!

Ø=Ý N E X T S T E P S

- **[CI/CD Pipelines Reference](cicd.md)** — Automate edge deployments using GitHub Actions.
- **[Observability & Time-Series Monitoring](monitoring.md)** — Stream console logs and check analytics datasets.

[SECTION: CI/CD PIPELINE AUTOMATION]

Ø=Ý C I / C D P I P E L I N E A U T O M A T I O N

Automating your deployment pipeline ensures that every commit pushed to your repository is systematically vetted for code style, type safety, syntax regressions, and unit test coverage before being deployed to your live production trading nodes.

This guide provides the complete specification and production-grade YAML workflow for implementing **GitHub Actions** integration pipelines.

Ø<ß×þ T H E C O N T I N U O U S I N T E G R A T I O N & D E P L O

The automation workflow executes a series of strict quality gates. A failure at any step halts the pipeline and blocks deployment:

```
[ G i t   P u s h :   m a i n ] % % %º   [ 1 .   C h e c k o u t   s u b m o d u l e s ] %

          % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
          %¼
[ 3 .   C o d e   F o r m a t t i n g   &   L i n t ] % % %º   [ 4 .   t s c   T y p e c h

          % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
          %¼
[ 6 .   S e q u e n c e d   E d g e   D e p l o y m e n t ] % % %º   [ 7 .   T e l e g r a
```

Ø=ÜÝ C O M P L E T E G I T H U B A C T I O N S W O R K F L O W (`

Create a YAML workflow file inside your repository at `.github/workflows/deploy.yml`:

```yaml
name: "Hoox Production CI/CD Pipeline"

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

```yaml
jobs:
  verify-and-deploy:
    name: "Verify & Deploy Stack"
    runs-on: "ubuntu-latest"
    timeout-minutes: 15

    steps:
      # 1. Checkout repository with all worker submodules recursively
      - name: "Checkout Codebase"
        uses: "actions/checkout@v4"
        with:
          submodules: "recursive"
          fetch-depth: 0

      # 2. Install Bun JavaScript runtime environment
      - name: "Setup Bun Runtime"
        uses: "oven-sh/setup-bun@v2"
        with:
          bun-version: "latest"

      # 3. Cache Bun dependency modules to optimize pipeline speeds
      - name: "Cache Workspace Dependencies"
        uses: "actions/cache@v4"
        with:
          path: "~/.bun/install/cache"
          key: "${{ runner.os }}-bun-${{ hashFiles('**/bun.lockb') }}"
          restore-keys: |
            ${{ runner.os }}-bun-

      # 4. Install all monorepo dependencies
      - name: "Install Dependencies"
        run: "bun install"

      # 5. Check formatting and ESLint rules
      - name: "Run Lint Checks"
        run: "bun run lint"

      # 6. Verify TypeScript compile-time type-safety (tsc --noEmit)
      - name: "Run Type Checks"
        run: "bun run typecheck"

      # 7. Execute all unit and integration test assertions (excluding live tests)
      - name: "Run Bun Test Suite"
        run: "bun test"

      # 8. Deploy all database schemas and workers in correct sequence
      - name: "Deploy Entire Edge Stack"
        if: "github.ref == 'refs/heads/main' && github.event_name == 'push'"
        env:
          CLOUDFLARE_API_TOKEN: "${{ secrets.CLOUDFLARE_API_TOKEN }}"
          CLOUDFLARE_ACCOUNT_ID: "${{ secrets.CLOUDFLARE_ACCOUNT_ID }}"
          SUBDOMAIN_PREFIX: "${{ secrets.SUBDOMAIN_PREFIX }}"
```

```
          run: |
            # Bootstrap local path binaries
            bun run build:cli

            # Deploy D1 SQL schemas to production
            npx wrangler d1 execute trade-data-db --file=workers/trade-worker/schema.sql
--remote --yes

            # Sequentially upload all enabled workers
            ./packages/cli/bin/hoox.js deploy all --auto --quiet

            # Post-deployment: update service bindings URLs globally
            ./packages/cli/bin/hoox.js deploy update-internal-urls --quiet

            # Post-deployment: apply KV manifest configurations
            ./packages/cli/bin/hoox.js deploy kv-config --quiet

            # Post-deployment: update Telegram bot webhook routing path
            ./packages/cli/bin/hoox.js deploy telegram-webhook --quiet
```

Ø=Ý M A N A G I N G S E C R E T S & V A R I A B L E S C O P E S

To authorize GitHub to interact with your Cloudflare account, navigate to your repository’s **Settings > Secrets and variables > Actions** and register three Action Secrets:

1. **`CLOUDFLARE_API_TOKEN`**: A secure, scoped token with Workers, D1, KV, and DNS write permissions.
2. **`CLOUDFLARE_ACCOUNT_ID`**: Your unique 32-character Cloudflare dashboard hash.
3. **`SUBDOMAIN_PREFIX`**: The subdomain namespace chosen for your deployments.

> **Note:** You **never** need to store worker-specific secrets (like Bybit API keys or Telegram Bot tokens) in GitHub Secrets. These are stored directly in Cloudflare's secured key vaults. The CI pipeline only deploys the code logic; the running edge isolates pull their credentials locally from Cloudflare’s hardware-level Secret Store at runtime.

Ø>Ýê P I P E L I N E C A C H I N G & C O N C U R R E N C Y C O N T

- **Concurrency Lock**: The `concurrency` block configured in the YAML ensures that if you push a new commit while a previous deployment pipeline is running, GitHub Actions will **automatically cancel** the older, redundant run to avoid race conditions or double-deploying.
- **Bun Cache**: Caching dependency directories reduces build-time installation overhead from minutes to **under 8 seconds**.

Ø=Ý N E X T S T E P S

- **[Production Deployment Manual](production.md)** — Audit DNS zones and custom domain routing options.
- **[System Observability & Monitoring](monitoring.md)** — Stream console logs and check analytics datasets.

[SECTION: OBSERVABILITY & TELEMETRY]

Ø=ÜÊ O B S E R V A B I L I T Y & T E L E M E T R Y

Operating a globally distributed algorithmic trading network demands extreme observability. A silent failure in an order loop or a transient API throttling event can lead to missed targets and unhedged exposures.

This guide outlines our **telemetry architecture**, covering Cloudflare Workers 100% request sampling, time-series SQL database queries, R2 logging, and automated alarm systems.

&¡ 1. 100% REQUEST HEAD SAMPLING

To monitor traffic at Cloudflare's edge compiler level, every worker in the Hoox monorepo enables native **observability tracing** inside its `wrangler.jsonc`:

```
{
  "observability": {
    "enabled": true,
    "head_sampling_rate": 1, // Captures and logs 100% of all incoming requests
  },
}
```

METRICS TRACKED AUTOMATICALLY

Once enabled, Cloudflare captures and graphs these metrics near-instantaneously:

- **Requests velocity**: Total hits and requests/second rates.
- **CPU execution duration (ms)**: The exact CPU time consumed by the isolate thread.
- **Uncaught Exceptions**: Crashes or code errors that bypass middlewares.
- **Subrequest Counts**: Outbound HTTP calls made to exchange APIs or other workers.

Ø=Ü¾ 2. CLOUDFLARE ANALYTICS ENGINE (SQL

While Cloudflare collects basic HTTP metrics, Hoox uses **Cloudflare Analytics Engine** inside `analytics-worker` to track trading-specific variables (e.g. execution slippage, exchange response delays, fee sums).

UNDER-THE-HOOD SQL QUERIES

The `analytics-worker` exposes a SQL interface to query the metrics dataset directly from

your Next.js Dashboard:

```
/* Query 1: Calculate the 95th percentile latency of Bybit API order fills */
SELECT
  quantiles(0.95)(double1) as p95_latency_ms,
  count() as total_executions
FROM hoox_telemetry
WHERE blob1 = 'trade-worker'
  AND blob2 = 'bybit'
  AND timestamp >= now() - INTERVAL '1' HOUR

/* Query 2: Aggregate error velocities across all edge isolates */
SELECT
  blob1 as worker_name,
  count() as error_count
FROM hoox_telemetry
WHERE blob3 = '0' -- Success = false
  AND timestamp >= now() - INTERVAL '6' HOUR
GROUP BY worker_name
```

Ø=ÜÝ 3. LOGGING & R2 STORAGE OFFLOADING

Because transactional databases like SQLite D1 have write limits, Hoox implements a **dual-tier logging policy**:

- **High-Value Data**: Transaction statuses and fills are written to your D1 `trades` table.
- **Verbose Telemetry**: Full, verbose JSON payload exchanges, network headers, and WebSocket stream records are serialized and saved to **R2 Storage** using date-based key paths:
  `logs/bybit/BTCUSDT/2026-05-19/order-18049284739.json`

This offloading reduces database write volumes by **up to 75%**, keeping your database compact, fast, and fully within free-tier limits.

Ø=Þ¨ 4. STANDARDIZED ALARM SYSTEMS

If a critical error or execution failure occurs (e.g., an order is rejected due to insufficient margin, or the kill switch is flipped due to drawdown limits):

1. The executing worker intercepts the exception using the shared error handler.
2. Direct V8 Service Binding pings `telegram-worker` instantly.
3. You receive an emergency alert on your phone.

```
  // Standard alarm notification trigger
  if (orderResponse.status === "Rejected") {
    await env.TELEGRAM_SERVICE.fetch("https://telegram-worker/alert", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "X-Internal-Auth-Key": env.INTERNAL_KEY_BINDING,
      },
      body: JSON.stringify({
        chatId: env.TELEGRAM_CHAT_ID_DEFAULT,
                m e s s a g e :  `Ø=Þ¨  < b > E m e r g e n c y  O r d e r  R e j e c t ! <
BTCUSDT\nReason: Insufficient Margin`,
      }),
    });
  }
```

> **Tip:** Need to tail live worker logs to debug a custom strategy? Run `hoox logs tail trade-worker` in your terminal to open a real-time WebSocket connection to Cloudflare’s global edge logging console!

Ø=Ý N E X T S T E P S

- **[Debugging Runbook](../development/debugging.md)** — Diagnose local and remote V8 exceptions using diagnostic profiles.
- **[Self-Healing & Repair](../guides/repair.md)** — Recover from degraded workers and provision missing bindings.

[SECTION: ZERO TRUST & SECURITY HARDENING]

Ø=Ý Z E R O T R U S T & S E C U R I T Y H A R D E N I N

While the Hoox dashboard features a highly secure, custom cookie-based authentication middleware, wrapping your dashboard and API endpoints inside **Cloudflare® Zero Trust (Access)** provides an enterprise-grade security perimeter.

By placing your deployment behind Cloudflare Access, you can enforce Multi-Factor Authentication (MFA), restrict access to specific GitHub/Google SSO identities, evaluate device posture, and drop malicious scanner payloads at the DNS level before they ever hit your workers.

Ø<ß×þ T H E Z E R O T R U S T P R O T E C T I V E B O U N D A R Y

[P u b l i c W e b B r o w s e r] % % %º [C l o u d f l a r e E d g e (W A F

%

(MFA & GitHub SSO Validation)

%

%¼

[Next.js Dashboard Isolate]

- **Zero Public Exposure**: The dashboard isolate does not evaluate public logins directly.
- **MFA Gate**: Users are intercepted by a secure Cloudflare authentication card at the nearest edge PoP.
- **Zero Cost**: Cloudflare's Zero Trust free tier includes up to 50 users, which is more than enough for a personal algorithmic trading desk.

&¡ 1 . S T E P - B Y - S T E P D A S H B O A R D A C C E S S S E T

STEP 1: ENABLE ZERO TRUST ON YOUR ACCOUNT

1. Log in to the Cloudflare Dashboard and click **Zero Trust** on the sidebar.
2. If this is your first time, follow the onboarding prompts to register a unique **Team Name** (e.g. `alpha-trading.cloudflareaccess.com`).

STEP 2: CREATE A SELF-HOSTED APPLICATION

1. In the Zero Trust dashboard, navigate to **Access > Applications** and click **Add an

application**.
2. Select **Self-hosted**.
3. **Application Name**: `Hoox Dashboard Cockpit`.
4. **Session Duration**: Select your preference (e.g. `24 Hours` to prevent constant login prompts).
5. **Application Domain**: Enter the custom domain mapped to your dashboard worker (e.g., `hoox.my-trading-empire.com`).

STEP 3: CONFIGURE AUTHORIZATION POLICIES

1. Click **Next** to proceed to the Policies tab.
2. **Policy Name**: `Allow Admin Only`.
3. **Action**: `Allow`.
4. **Configure Rules**:
   - **Include**: Select **Emails** and enter your personal email address (enables Email OTP).
   - **Include (SSO)**: Alternatively, select **GitHub Org/Teams** or **Google Workspace** to enable SSO integrations.
5. In the **Require** block, you can optionally require a valid **security key (MFA)** or **device posture check** (e.g. verifying that your laptop runs a specific OS version).

STEP 4: MAP IDENTITY PROVIDERS & SAVE

1. In **Settings > Authentication**, link your desired login providers (Google Workspace, GitHub OAuth, or Email OTP).
2. Save the application.
3. Open your browser and navigate to your custom domain (`https://hoox.my-trading-empire.com`). You will be intercepted by your Cloudflare Access card. Once authorized, you are passed cleanly to your Next.js dashboard.

Ø>Ýñ 2. S T R I C T W A F W E B H O O K I P A L L O W - L I S T

To ensure that **only** TradingView's official servers can fire signals to your `/webhook` entryway:

1. Under your Cloudflare DNS zone dashboard, navigate to **Security > WAF > Custom Rules**.
2. Click **Create Rule**.
3. **Rule Name**: `Restrict /webhook to TradingView IPs`.
4. **Field**: `URI Path` | **Operator**: `equals` | **Value**: `/webhook`.

5. **And**: `IP Source Address` | **Operator**: `is not in` | **Value**: (Paste TradingView's official IP ranges here, which are automatically synced by running the `hoox waf configure --TradingView-only` command).
6. **Action**: **Block** (or **Challenge**).
7. Save. All unauthorized traffic hitting `/webhook` is dropped instantly at the DNS edge, preventing any V8 compute load.

```
# Automated WAF setup via CLI
hoox waf configure --TradingView-only
```

&™þ 3 . O P T I O N A L : B Y P A S S I N G L O C A L D A S H B O

Once your custom domain is wrapped inside Cloudflare Access, the dashboard's built-in login form (`DASHBOARD_USER`, `DASHBOARD_PASS`) becomes redundant.

To streamline access:

1. Edit the Next.js `middleware.ts` file inside `workers/dashboard/src/`.
2. Toggle the authentication checker to leverage Cloudflare's Access headers:
   ```typescript
   // Next.js Edge Middleware
   export function middleware(request: Request) {
     // Verify the JWT payload injected in headers by Cloudflare Access
     const cfAccessJwt = request.headers.get("Cf-Access-Jwt-Assertion");
     if (cfAccessJwt) {
       // Cloudflare has already authenticated the session. Bypass local login.
       return NextResponse.next();
     }
     // Fallback to local cookie checks...
   }
   ```

Ø=Ý N E X T S T E P S

- **[Next.js Dashboard worker Profile](../workers/dashboard.md)** — Review OpenNext compilation and asset bindings.
- **[System Observability & Metrics](monitoring.md)** — Setup time-series logging and Analytics Engine tables.

[SECTION: LOCAL DEVELOPMENT SETUP]

Ø=Ü» L O C A L D E V E L O P M E N T S E T U P

Hoox provides a comprehensive local development workspace designed to emulate your production Cloudflare edge environment. Developers can choose between running microservices natively on local Wrangler V8 isolates or inside completely isolated, containerized Docker stacks, with real-time log tailing and TUI diagnostics.

&¡ 1. D E V R U N T I M E S E L E C T I O N : N A T I V E V S .

The `hoox dev start` command orchestrates the boot sequence for all enabled workers and supports two execution runtimes:

Runtime Mode	Boot Command	Latency / Hot-Reload	Isolation Level	Ideal Use Case
Native	`wrangler dev`	**< 10ms**	Low (shares local host ports)	High-speed script tweaking, rapid feature iterations.
Docker	`docker compose up`	**~100ms**	High (isolated Linux containers)	Testing full integrations, database migrations, queue failovers.

THE GUIDED STARTUP FLOW

When you run `hoox dev start`:

1. **Wrangler Version Verification**: Probes your global/local Wrangler package. If Wrangler is outdated, it prompts an advisory upgrade warning:
   ```
   & þ W r a n g l e r C L I i s o u t d a t e d ( v 3 . 5 0 . 0 < v 3 . 8 8
    Run `bunx wrangler update` to update, or press Enter to continue.
   ```
2. **Docker Environment Probing**: Checks if the Docker daemon is active and `docker-compose.yml` is present in the workspace.
3. **Runtime Preference Lock**: If both runtimes are available, the CLI prompts you to select your preference. This choice is written to your `wrangler.jsonc` file under the `"dev": { "runtime": "..." }` block, bypassing future prompts.

```
# Override the saved runtime preference on the fly
hoox dev start --runtime native
hoox dev start --runtime docker
```

Ø=Ü3 2. DOCKER COMPOSE ORCHESTRATION & P

For full stack containerized development, Hoox divides operations into three Docker Compose profiles in your root `docker-compose.yml`:

```
# Profile 1: Spin up all background workers only
docker compose --profile workers up

# Profile 2: Spin up the Next.js frontend only
docker compose --profile dashboard up

# Profile 3: Full Stack (All workers + Next.js dashboard)
docker compose --profile full up
```

Ø=ÜÍ 3. LOCAL PORT MAPPING REGISTRY

In the local environment, each V8 dev instance mounts to a dedicated host port. Ensure no other applications are occupying these ports before launching:

Microservice	Default Port	Local Binding URL	Purpose
`hoox`	`8787`	`http://localhost:8787`	Ingress Gateway
`trade-worker`	`8789`	`http://localhost:8789`	Order Execution Engine
`telegram-worker`	`8791`	`http://localhost:8791`	Notifications Hub
`d1-worker`	`8792`	`http://localhost:8792`	Relational SQLite Manager
`web3-wallet`	`8793`	`http://localhost:8793`	On-Chain DeFi Node
`dashboard`	`8794`	`http://localhost:8794`	Next.js Dashboard SSR
`agent-worker`	`8795`	`http://localhost:8795`	Cron Risk Monitor

Ø=Þáþ 4. LOCAL ENVIRONMENTAL MOCKING (`.

Because production secrets are locked inside Cloudflare's secured key vaults, Wrangler dev uses local `.dev.vars` files located inside each worker's directory to mock API keys

and credentials:

```
# 1. Copy the secure template
cp workers/hoox/.dev.vars.example workers/hoox/.dev.vars

# 2. Inject local mock keys for testing
echo 'INTERNAL_KEY_BINDING="local_test_key"' >> workers/hoox/.dev.vars
```

> **Warning:** `.dev.vars` files contain local plaintext keys and are excluded from git index tracking via `.gitignore`. **Never** remove these files from `.gitignore` or check them into your repository.

> **Tip:** If local tests fail with `Time-Stamp Expired` errors when calling exchange APIs, check that your local machine's NTP system clock is synchronized: `sudo ntpdate pool.ntp.org` (or check date/time settings on macOS/Windows)!

Ø=Ý NEXT STEPS

- **[Testing Standards](testing.md)** — Run unit and integration tests using Bun's native test runner.
- **[Debugging Runbook](debugging.md)** — Debug local and remote isolates, tail logs, and trace queries.

[SECTION: TESTING FRAMEWORK & QA STANDARDS]

Ø>Ýê T E S T I N G F R A M E W O R K & Q A S T A N D A R

To protect live capital and ensure robust order routing, Hoox mandates a rigorous testing pipeline. With money on the line, we verify every contract calculation, rate-limiting gate, and database query.

Our test suite is powered natively by **Bun's high-speed test runner**, comprising **106 test files** and **1,574 individual test assertions** split into four distinct diagnostic layers.

Ø<ßšþ T H E 4 Q A T E S T I N G L A Y E R S

[U n i t T e s t s (1 , 4 5 8 A s s e r t i o n s)] % % %º M o c k V 8 middlewares.

%

[I n t e g r a t i o n T e s t s (3 4 A s s e r t i o n s)] % % %º M i n i stacks).

%

[E 2 E S m o k e T e s t s (5 A s s e r t i o n s)] % % %º C L I l i traps.

%

[L i v e R e s o u r c e T e s t s (7 7 A s s e r t i o n s)] % % %º R e execution.

&¡ R U N N I N G T E S T S : C L I C O M M A N D S

A. CORE PLATFORM VERIFICATION (EXCLUDING LIVE)

```
# 1. Run all unit, integration, and E2E smoke tests in parallel
bun test

# 2. Run the suite and output a detailed V8 coverage report
bun test --coverage
```

B. WORKSPACE-SPECIFIC TARGETED RUNS

To optimize developer feedback loops, you can target specific workspaces or workers:

```
# Run CLI commands tests only (packages/cli/)
bun run test:cli
```

```
# Run Terminal UI tests only (packages/tui/)
bun run test:tui

# Run shared helper tests only (packages/shared/)
bun run test:shared

# Run all edge workers unit tests (workers/*)
bun run test:workers

# Run a single specific test file with hot-reload watch mode
bun test workers/agent-worker/src/index.test.ts --watch
```

C. ADVANCED INTEGRATION & LIVE RUNS

```
# Run Miniflare 3 gateway integration tests
bun run test:integration

# Run E2E CLI lifecycle smoke tests
bun run test:e2e

# Run live Cloudflare API integration tests (requires tests/live/.env credentials)
bun run test:live --jobs 1
```

Ø=Ý T Y P E - S A F E M O C K I N G S P E C I F I C A T I O N S (N

To enforce strict TypeScript compiler safety, test files **must never** utilize `as any` to bypass types when mock-binding resources. Always cast stubs using `as unknown as Env`:

```
import { describe, it, expect } from "bun:test";
import type { Env } from "../src/index";

describe("trade-worker Gateway Router Mocking", () => {
  it("should securely mock internal service binding fetchers", async () => {
    // 1. Construct a type-safe mock environment structure
    const mockEnv = {
      INTERNAL_KEY_BINDING: "local_secret_token_183",
      TELEGRAM_SERVICE: {
        fetch: async (url: string, init?: RequestInit) => {
          // Verify auth headers exist
          const headers = init?.headers as Record<string, string>;
          if (headers["X-Internal-Auth-Key"] !== "local_secret_token_183") {
            return new Response(JSON.stringify({ success: false }), {
              status: 401,
            });
          }

          return new Response(
            JSON.stringify({
```

```
              success: true,
              messageId: 4829,
            }),
            { status: 200 }
          );
        },
      } as Fetcher,
    } as unknown as Env;

    // 2. Execute assertions
    const res = await mockEnv.TELEGRAM_SERVICE.fetch(
      "https://telegram-worker/alert",
      {
        method: "POST",
        headers: {
          "X-Internal-Auth-Key": "local_secret_token_183",
        },
      }
    );

    const data = await res.json();
    expect(res.status).toBe(200);
    expect(data.success).toBe(true);
    expect(data.messageId).toBe(4829);
  });
});
```

Ø=Þ¢ CONTINUOUS INTEGRATION GATES & COVE

Our GitHub Actions workflow enforces two strict quality gates before any code is approved for production deployment:

1. **TypeScript Type Safety**: All workspaces must compile without errors using `tsc --noEmit`.
2. **Coverage Thresholds**: The monorepo enforces a **minimum 80% coverage threshold** across all core execution paths (`packages/cli`, `packages/shared`, `workers/hoox`, `workers/trade-worker`).

```
# Check your local workspace coverage statistics
bun test packages/shared/ --coverage
```

Ø=Ý NEXT STEPS

- **[Debugging Telemetry Runbook](debugging.md)** — Learn how to trace active V8 memory, tail logs, and audit SQL execution.
- **[Local Development Setup](local-dev.md)** — Configure Wrangler and Docker compose to run testbeds.

[SECTION: EDGE DEBUGGING & TELEMETRY]

Ø=Ü E D G E D E B U G G I N G & T E L E M E T R Y

Debugging globally distributed serverless microservices presents unique operational challenges. Because code runs on Cloudflare's edge V8 isolates without a persistent server host, traditional step-through debugging is replaced by **structured logging, real-time edge telemetry tailing, and distributed tracing**.

This document is your engineering runbook for debugging and resolving runtime anomalies in the Hoox monorepo.

&¡ 1 . R E A L - T I M E T E L E M E T R Y : T A I L I N G P R O D

Cloudflare’s **`wrangler tail`** engine establishes an active, secure WebSocket stream straight from Cloudflare's global edge nodes to your terminal, printing every `console.log`, exception trace, and HTTP status code in real-time.

You can trigger this streaming telemetry via the Hoox CLI or Wrangler:

```
# A. Recommended: Stream live console logs for a specific worker via Hoox CLI
hoox logs tail trade-worker

# B. Stream gateway traffic in real-time
hoox logs tail hoox

# C. Alternatively, invoke Wrangler directly for custom configurations
npx wrangler tail hoox --config workers/hoox/wrangler.jsonc
```

Ø=ÜÝ 2 . D I S T R I B U T E D T R A C I N G : T H E ` R E Q U E S

B e c a u s e a s i n g l e T r a d i n g V i e w a l e r t f l o w s t h r o u g h !’ d 1 - w o r k e r !’ t e l e g r a m - w o r k e r) , l o c a t i n g a s p e c i concurrent entries is impossible without a unified trace parameter.

THE REQUESTID PROTOCOL

1. **Generation**: The `hoox` gateway generates a unique, cryptographically secure **UUIDv4** immediately upon receiving a webhook payload. This is set as the `requestId` in the transaction context.
2. **Propagation**: Every internal service binding call transmits this UUID as the `X-Request-Id` HTTP header.
3. **Structured Ingestion**: When logging error telemetry or writing trade fills to R2

and D1 databases, workers attach the `requestId` as a dedicated index column.
4. **Unified Auditing**: You can audit an entire transaction's lifecycle across all 9 workers by running a single database query filtering by the trace ID:

```
SELECT created_at, worker, message, severity
FROM system_logs
WHERE request_id = '9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d'
ORDER BY created_at ASC
```

Ø=Þáþ 3. O P E R A T I O N A L R U N B O O K S F O R C O M M O N

A. SYMPTOM: `401 UNAUTHORIZED` ON INTERNAL WORKER CALLS

- **Diagnostics**: The calling worker gets a `401 Unauthorized` response when querying internal services like `d1-worker` or `trade-worker`.
- **Primary Root Cause**: The `INTERNAL_KEY_BINDING` secret does not match between the calling worker and the target worker.
- **Resolution**:
  1. Inspect the secret existence on both workers: `hoox secrets check`.
  2. If mismatched, re-inject the secret globally using one command:
     ```bash
     hoox secrets set INTERNAL_KEY_BINDING "your_secure_shared_internal_key"
     ```

B. SYMPTOM: `502 BAD GATEWAY` ON WEBHOOK ROUTES

- **Diagnostics**: An incoming signal returns a 502 error instantly.
- **Primary Root Cause**: A Service Binding target declared in the gateway's `wrangler.jsonc` has not been deployed yet.
- **Resolution**:
  1. Run the dependency check: `hoox check health`.
  2. Redeploy the entire stack in the correct dependency order:
     ```bash
     hoox deploy all --auto
     ```

C. SYMPTOM: `D1_ERROR: NO SUCH TABLE`

- **Diagnostics**: Worker database transactions fail with table missing errors.
- **Primary Root Cause**: Relational SQLite schemas were never initialized on your

production D1 instance.
- **Resolution**:
  1. Initialize database tables and run pending drizzle migrations:
     ```bash
     hoox db apply --remote
     hoox db migrate --remote
     ```

D. SYMPTOM: KV CONFIGURATION PROPAGATION DELAYS

- **Diagnostics**: You toggled a KV configuration (e.g. turned `trade:kill_switch = false`), but some incoming signals are still being rejected.
- **Primary Root Cause**: Cloudflare KV is eventually consistent. Updates can take up to 10 seconds to propagate to all global edge locations.
- **Resolution**: Wait 10-15 seconds for cache validation to settle globally. Do not trigger rapid test webhooks immediately after modifying configuration keys.

Ø=Ý N E X T S T E P S

- **[Testing Framework & QA Standards](testing.md)** — Run local unit and integration tests using Bun's native test runner.
- **[Self-Healing & System Repair](../guides/repair.md)** — Run diagnostics, targeted component repairs, and full system rebuilds.

[SECTION: API ENDPOINT DIRECTORY]

Ø=Ý A P I E N D P O I N T D I R E C T O R Y

This directory provides the complete HTTP REST API specification for all public and internal endpoints exposed across the Hoox edge microservices stack.

Ø=Ý S E C U R I T Y & A U T H O R I Z A T I O N H E A D E R S

All internal calls between workers (via V8 Service Bindings) must provide the standard internal authentication header:

```
X-Internal-Auth-Key: <INTERNAL_KEY_BINDING>
```

Public webhook and dashboard endpoints use cookie authorization or custom API keys:

- **Webhook Ingestion**: Authenticated using the `apiKey` property inside the JSON payload.
- **Telegram webhook**: Authenticated via the secure token path: `/telegram/<TELEGRAM_WEBHOOK_SECRET>`.

Ø=Þ€ I N G R E S S W E B H O O K E N D P O I N T S (` W O R K E R S

The public gateway acts as the primary firewall and entry entryway for all external trade signals.

A. INGEST SIGNAL WEBHOOK

Receives TradingView alerts or automated cURL signals.

- **Path**: `/webhook`
- **Method**: `POST`
- **JSON Payload**:

```json
{
  "apiKey": "secure_webhook_key_18305",
  "exchange": "bybit",
  "action": "LONG",
  "symbol": "BTCUSDT",
  "quantity": 0.002,
  "leverage": 10,
  "idempotencyKey": "uuid-9b1deb4d-3b7d"
```

```
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "exchange": "bybit",
    "symbol": "BTCUSDT",
    "action": "LONG",
    "result": {
      "orderId": "18049284739",
      "status": "Filled",
      "price": 68425.5
    }
  }
  ```

B. PROACTIVE HEALTH DIAGNOSTICS

- **Path**: `/health`
- **Method**: `GET`
- **Success Response (200 OK)**:
  ```json
  {
    "status": "ok",
    "timestamp": 1779261050000,
    "bindings": {
      "d1": "connected",
      "kv": "connected",
      "queue": "active"
    }
  }
  ```

ø=ÝÄþ D A T A B A S E S E R V I C E E N D P O I N T S (` W O R K E

The `d1-worker` serves as a private, internal SQL proxy database.

A. EXECUTE SINGLE SQL STATEMENT

- **Path**: `/query`

- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "query": "SELECT * FROM trades WHERE symbol = ? LIMIT ?",
    "params": ["BTCUSDT", 5]
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "results": [{ "id": "trade-1", "symbol": "BTCUSDT", "price": 68400 }],
    "meta": { "rows_read": 1, "duration": 3.5 }
  }
  ```

B. EXECUTE TRANSACTIONAL BATCH STATEMENTS

- **Path**: `/batch`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "queries": [
      {
        "query": "INSERT INTO trades (id, symbol) VALUES (?, ?)",
        "params": ["t-1", "ETHUSDT"]
      },
      {
        "query": "UPDATE positions SET size = size + 0.05 WHERE symbol = 'ETHUSDT'",
        "params": []
      }
    ]
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "results": [{ "meta": { "changes": 1 } }, { "meta": { "changes": 1 } }]
  }
  ```

Ø>Ýà A I R I S K & C H A T E N D P O I N T S (` W O R K E R S /

Bridges background risk audits and multi-provider AI chat streams.

A. CONVERSATIONAL CHAT STREAM (SERVER-SENT EVENTS)

- **Path**: `/agent/chat`
- **Method**: `POST`
- **Headers**: `Accept: text/event-stream`
- **JSON Payload**:
  ```json
  {
    "prompt": "Summarize my trade history and average win rate today.",
    "stream": true,
    "provider": "anthropic"
  }
  ```
- **Success Response**: Emits standard text/event-stream updates, terminating with `data: [DONE]`.

B. MULTIMODAL AI VISION AUDIT

- **Path**: `/agent/vision`
- **Method**: `POST`
- **JSON Payload**:
  ```json
  {
    "imageBase64": "iVBORw0KGgoAAAANSUhEUgAA...",
    "prompt": "Evaluate this trading chart image for support and resistance levels."
  }
  ```
- **Success Response (200 OK)**:
  ```json
  {
    "success": true,
    "analysis": "Based on the provided chart screenshot, there is strong horizontal support at $67,200..."
  }
  ```

Ø=Ý N E X T S T E P S

- **[Request Payload Schemas](payloads.md)** — Analyze the complete JSON request schemas and type rules.
- **[Standard Response Schemas](responses.md)** — Check error factories and normal JSON envelopes.

[SECTION: REQUEST PAYLOAD SCHEMAS]

Ø=Üæ R E Q U E S T P A Y L O A D S C H E M A S

To maintain robust data routing and prevent runtime failures, Hoox enforces a strict **payload contract** across all internal and public service boundaries. All incoming requests are validated by active JSON Schema or Zod middleware before entering V8 execution loops.

This document details the exact JSON schemas, TypeScript interfaces, and validation rules for all primary request payloads.

Ø=Þáþ 1 . S T A N D A R D R E Q U E S T E N V E L O P E

Every service-to-service invocation (via Service Bindings) wraps its business parameters inside a standardized **Request Envelope** containing distributed tracing indices:

```
export interface RequestEnvelope<T> {
  requestId: string; // Unique UUIDv4 distributed trace ID
  payload: T; // Service-specific payload
}
```

```
{
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "payload": {
    "symbol": "BTCUSDT",
    "quantity": 0.005
  }
}
```

Ø=ÜÈ 2 . T R A D E E X E C U T I O N P A Y L O A D S (` T R A D E

The execution engine processes two primary types of order payloads:

A. CENTRALIZED EXCHANGE ORDER PAYLOAD (`/WEBHOOK` & `/PROCESS`)

```
export interface CexOrderPayload {
  exchange: "binance" | "bybit" | "mexc"; // Target exchange
  action: "LONG" | "SHORT" | "CLOSE"; // Margin position action
  symbol: string; // Standardized asset ticker (e.g. "BTCUSDT")
  quantity: number; // Order size in contracts or tokens
  leverage?: number; // Optional margin leverage multiplier
  price?: number; // Optional execution limit price
  orderType?: "MARKET" | "LIMIT"; // Default: MARKET
```

```
}

{
  "exchange": "bybit",
  "action": "LONG",
  "symbol": "BTCUSDT",
  "quantity": 0.01,
  "leverage": 10,
  "orderType": "MARKET"
}
```

B. ON-CHAIN DEFI SWAP PAYLOAD (`/DEX`)

```
export interface DexSwapPayload {
  chain: "ethereum" | "arbitrum" | "polygon"; // EVM target network
  to: string; // Recipient address or Uniswap Router
  value: string; // Transaction value in Wei (as string)
  data: string; // Encoded contract call data bytes
  gasLimit?: number; // Optional transaction gas limit
}

{
  "chain": "arbitrum",
  "to": "0x6b175474e89094c44da98b954eedeac495271d0f",
  "value": "100000000000000000",
  "data": "0xa9059cbb000000000000000000000000..."
}
```

Ø=ÝÄþ 3 . D A T A B A S E S Q L Q U E R Y P A Y L O A D S (` D

The SQLite database proxy accepts parameterized SQL statements to protect against SQL injections:

A. EXECUTE SINGLE SQL STATEMENT

```
export interface SqlQueryPayload {
  query: string; // Parameterized SQL statement
  params: Array<string | number | boolean | null>; // Binding values
}

{
  "query": "SELECT * FROM trades WHERE symbol = ? AND status = ?",
  "params": ["BTCUSDT", "Filled"]
}
```

Ø=Ü¬ 4 . T E L E G R A M N O T I F I C A T I O N P A Y L O A D S (

Used to push structured alerts and charts to your mobile device:

```
export interface TelegramAlertPayload {
  chatId: string; // Target Telegram Chat ID
  message: string; // Formatted message content
  parseMode?: "HTML" | "MarkdownV2"; // Default: HTML
}
```

```
{
  "chatId": "987654321",
  "message": "<b>Alert:</b> Position filled at 68,420.",
  "parseMode": "HTML"
}
```

> **Tip:** Adding new fields to a payload schema? Remember to update the corresponding type interfaces in `@jango-blockchained/hoox-shared/types` to ensure complete type safety across all workers!

Ø=Ý N E X T S T E P S

- **[API Endpoint Directory](endpoints.md)** — Analyze REST routes, headers, and endpoints mappings.
- **[Standard Response Schemas](responses.md)** — Check success envelopes and error models.

[SECTION: STANDARD RESPONSE SCHEMAS]

Ø=Üè S T A N D A R D R E S P O N S E S C H E M A S

To ensure predictable error handling and parsing across all microservices, Hoox enforces a **strict response contract**. Every worker responds with a standardized JSON envelope containing tracing details, operation status, results payload, and detailed error models.

This document details the exact JSON templates, types, and error factories utilized in the monorepo.

Ø=Þáþ 1 . S T A N D A R D R E S P O N S E E N V E L O P E

Every HTTP response returned by an edge worker is encapsulated within a unified JSON envelope:

```
export interface ResponseEnvelope<T> {
  success: boolean; // Absolute state of the operation
  requestId: string; // UUIDv4 trace ID mapped from request
  result?: T; // Operation success metadata payload
  error?: {
    // Detail model present only if success = false
    code: string; // Unique machine-parseable error token
    message: string; // Human-readable error description
    details?: any; // Optional specific array of field issues
  };
}
```

Ø<ßÆ 2 . S U C C E S S R E S P O N S E T E M P L A T E S

A. TRADE EXECUTION FILL SUCCESS (`TRADE-WORKER` - 200 OK)

```
{
  "success": true,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "result": {
    "orderId": "18049284739",
    "status": "Filled",
    "executedQty": 0.005,
    "price": 68425.5,
    "timestamp": 1779261050000
  },
  "error": null
}
```

B. TELEGRAM PUSH SUCCESS (`TELEGRAM-WORKER` - 200 OK)


```
{
  "success": true,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "result": {
    "messageId": 482904,
    "delivered": true
  },
  "error": null
}
```

Ø=Þ¨ 3. ERROR MODELS & EDGE ERROR CODES

When a worker operation fails, the `success` flag is set to `false`, the `result` field is omitted or set to `null`, and the `error` model is fully populated:

A. 400 BAD REQUEST (JSON VALIDATION ERROR)


```
{
  "success": false,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid JSON payload structure.",
    "details": [{ "field": "quantity", "issue": "Must be greater than zero." }]
  }
}
```

B. 409 CONFLICT (IDEMPOTENCY MUTEX INTERCEPT)


```
{
  "success": false,
  "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "error": {
    "code": "DUPLICATE_REQUEST",
    "message": "Order rejected. Durable Object locked: trace ID already processed in
the last 24 hours."
  }
}
```

Ø=Þàþ 4 . S H A R E D ` E R R O R S ` F A C T O R Y M I D D L E W

To enforce this response contract without writing redundant JSON wrappers in every worker, we use the standardized `Errors` factory from the shared monorepo package `@jango-blockchained/hoox-shared/errors`:

```
import { Errors } from "@jango-blockchained/hoox-shared/errors";

// 1. Validation Failures (400 Bad Request)
if (!payload.symbol) {
  return Errors.badRequest("Symbol is a required parameter.");
}

// 2. Secret Key Mismatch (401 Unauthorized)
if (requestHeaderKey !== env.INTERNAL_KEY_BINDING) {
  return Errors.unauthorized("Access Denied: Invalid X-Internal-Auth-Key.");
}

// 3. System Exceptions (500 Internal Server Error)
try {
  await database.write(data);
} catch (err: any) {
  return Errors.internal(`Database write exception: ${err.message}`);
}
```

These factory methods automatically serialize the exception, attach the active `requestId` from the context, set the correct HTTP status code, and return a compliant `Response` object.

Ø=Ý N E X T S T E P S

- **[API Endpoint Directory](endpoints.md)** — Analyze REST routes, headers, and endpoints mappings.
- **[Request Payload Schemas](payloads.md)** — Check JSON request payload specifications and typescript interfaces.

[SECTION: CLI ARCHITECTURE & FEATURES]

Ø=Þàþ C L I A R C H I T E C T U R E & F E A T U R E S

The **`hoox` CLI** (`packages/cli`) is the unified lifecycle engine of the trading platform monorepo. It governs local sandboxes, compiles TypeScript structures, coordinates Cloudflare infrastructure resources, deploys edge isolates, manages encrypted secrets, and executes self-healing diagnostics.

Ø<ß×þ M O N O R E P O W O R K S P A C E D E S I G N

The CLI is integrated into our monorepo using **Bun Workspaces**, which link local packages together. This design ensures that the CLI binary can resolve and load local shared types (`@jango-blockchained/hoox-shared`) and TUI components (`packages/tui`) instantly without network downloads or pre-compilation overhead:

```
hoox-setup/ (Monorepo Root)
  % % %   p a c k a g e s /
  %       % % %   c l i /              #  C L I  S o u r c e  c o d e  ( e n t r y  b i
  %       % % %   t u i /              #  O p e n T U I  d a s h b o a r d  s o u r c e
  %       % % %   s h a r e d /        #  C o m m o n  l i b r a r i e s  ( a u t h ,  r
  % % %   w o r k e r s /
  %       % % %   . . .                #  E d g e  V 8  i s o l a t e s
  % % %   p a c k a g e . j s o n      #  R o o t  w o r k s p a c e  m a n a g e r
```

&¡ 1 . C O M M A N D - L I N E C O R E A R C H I T E C T U R E S

The CLI binary parses terminal instructions using the following architectural layers:

A. COMMAND DISPATCHER (`PACKAGES/CLI/SRC/INDEX.TS`)

Intercepts all incoming arguments (e.g. `hoox infra provision`), evaluates global flags (`--json`, `--quiet`), validates configuration integrity, and delegates execution to target command modules under `src/commands/`.

B. CLOUDFLARE API ADAPTERS (`SRC/ADAPTERS/`)

Translates command intentions (like `hoox infra d1 create`) into parameterized Cloudflare API REST payloads, bypassing wrangler wrappers when high-performance execution is needed.

C. STATE ENGINE (`SRC/CORE/`)

Tracks active project profiles, enabled worker matrices, and workspace configuration formats (`wrangler.jsonc` vs. `wrangler.toml`).

Ø=Ý 2. DECLARATIVE CONFIG MAPPING & VAL

To prevent configuration drift, the CLI enforces strict type validation on `wrangler.jsonc` files:

1. **Config Interfaces**: Built-in parsers validate configuration files against type-safe TypeScript interfaces (`Config` and `WorkerConfig`) defined in `src/core/types.ts`.
2. **Setup Verification (`hoox check setup`)**: Compares active workspace profiles against example files, audits environment variables keys, and scans for missing bindings, outputting formatted terminal reports.

Ø=ÞÜ 3. SELF-HEALING & DIAGNOSTICS ENGIN

One of the CLI's most powerful features is its **guided repair framework** (`hoox repair` command groups):

- **Diagnostic Probes**: The `hoox repair check` command runs a 5-step checklist (verifying submodule checkouts, NPM/Bun package resolutions, TypeScript variables, Cloudflare zone links, and encrypted credentials).
- **Automated Recovery**: If a missing resource is identified (e.g. a D1 database ID is bound in `wrangler.jsonc` but the database doesn't exist on your Cloudflare account), the repair engine prompts you and provisions it automatically:

```
# Provision missing Cloudflare bindings and repair system states
hoox repair infra
```

> **Tip:** Every single command supports the `--json` global flag. This outputs machine-parseable JSON payloads (e.g. `hoox monitor status --json`), allowing you to integrate the CLI with external telemetry dashboards or alert scripts effortlessly!

Ø=Ý NEXT STEPS

- **[CLI Reference Manual](../../enduser/reference/cli-commands.md)** — Review the complete command tree, positional arguments, and flags.
- **[Wrangler Setup & Tooling](development/local-dev.md)** — Configure Wrangler to bind local D1 and KV instances for dev testing.